



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Inteligencia Artificial

Sistema de accesibilidad educativa en tiempo real con transcripción adaptativa: comparativa de técnicas sobre modelos ASR en español

Trabajo Fin de Estudios

presentado por: Jesús Jiménez Alcántara

Dirigido por: [Pendiente de asignación]

Ciudad: Vélez-Málaga

Fecha: Marzo de 2027

Índice de Contenidos

Resumen	VIII
Abstract	IX
1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del trabajo	2
1.3. Aportaciones	2
1.4. Estructura del trabajo	3
2. Contexto y Estado del Arte	5
2.1. El problema: subtitular una clase para quien no puede seguirla	5
2.1.1. El marco normativo	5
2.1.2. Para quién funciona un subtítulo	5
2.2. Reconocimiento automático del habla de propósito general	7
2.2.1. Una familia alternativa: los transductores	7
2.3. Técnicas de adaptación al dominio	8
2.4. Recursos de habla en español	8
2.4.1. El corpus objetivo	9
2.4.2. Los recursos efectivamente empleados	9
2.4.3. La calidad de la referencia como variable	10
2.5. Evaluación de sistemas de reconocimiento	10
2.5.1. La métrica habitual y sus límites	10
2.5.2. Alucinaciones	11
2.6. Subtitulado en tiempo real	11
2.7. Posicionamiento del trabajo	12
3. Identificación de Requisitos	13
3.1. Requisitos del sistema de subtitulado	13
3.1.1. Funcionales	13
3.1.2. No funcionales	14
3.1.3. Legales y de protección de datos	15

3.2.	Requisitos del procedimiento experimental	16
3.3.	Trazabilidad de los requisitos	17
4.	Objetivos	18
4.1.	Objetivo general	18
4.2.	Objetivos específicos	18
4.3.	Criterios de cumplimiento	19
4.4.	Alcance y exclusiones	20
5.	Desarrollo del trabajo	21
5.1.	Metodología experimental	21
5.1.1.	Principios de diseño	21
5.1.2.	Potencia estadística requerida	22
5.1.3.	Corpus	22
5.2.	Línea base sin adaptación	23
5.2.1.	Elección del modelo	23
5.2.2.	El modelo elegido y por qué no es el más rápido	25
5.3.	Contraste con una arquitectura distinta	25
5.4.	Comparativa de técnicas de adaptación	27
5.4.1.	Prompting contextual	27
5.4.2.	Post-corrección con un modelo de lenguaje local	28
5.4.3.	Ajuste fino con LoRA	29
5.5.	El WER no basta para evaluar accesibilidad	29
5.6.	Desarrollo de la aplicación	31
5.6.1.	Arquitectura	31
5.6.2.	La pila tecnológica y el puente entre los dos núcleos	32
5.6.3.	Los componentes del núcleo	32
5.6.4.	Troceado del audio: latencia frente a calidad	35
5.6.5.	Requisito de hardware	38
5.6.6.	Robustez frente al ruido de aula	38
5.6.7.	Contención de alucinaciones en el sistema desplegado	40
5.6.8.	La interfaz	41
5.6.9.	Latencia del circuito completo	41
5.6.10.	Verificación de la aplicación	42

5.6.11. Despliegue y portabilidad	43
5.6.12. Incorporación de la técnica ganadora	44
5.7. Incidentes metodológicos	44
6. Conclusiones y Trabajo Futuro	47
6.1. Conclusiones	47
6.1.1. Sobre las técnicas de adaptación	47
6.1.2. Sobre dónde está realmente el margen de mejora	47
6.1.3. Sobre la evaluación	48
6.1.4. Sobre la elección del modelo base	48
6.1.5. Sobre la validez de las mediciones	49
6.2. Verificación de los objetivos	49
6.3. Limitaciones	51
6.4. Trabajo futuro	52
Referencias	54
A. Reproducibilidad	58
A.1. Entorno de ejecución	58
A.2. Trazabilidad de los resultados	58
A.3. Identidad de un resultado	61
A.4. Generación de tablas y figuras	61
A.5. Verificaciones previas	61
A.6. Estructura del repositorio	62
B. Registro de riesgos	63
B.1. Riesgos previstos en la planificación inicial	63
B.2. Riesgos detectados midiendo	63
C. Despliegue y uso del sistema	67
C.1. Qué hace falta antes de empezar	67
C.2. Cómo se distribuye el sistema	71
C.3. Puesta en marcha	76
C.4. Incorporar el modelo adaptado	79
C.5. Qué hacer si algo no funciona	79
C.6. Arranque sin contenedores	80

Índice de Ilustraciones

5.1. Calidad frente a coste y rendimientos decrecientes por tamaño de modelo. La línea vertical marca el límite de tiempo real. Generada desde los datos de ejecución.	23
5.2. Topología del sistema desplegado. El reconocimiento ocurre una sola vez en el servidor con independencia del número de dispositivos conectados; el alumnado solo visualiza, sin instalar nada ni conceder permisos.	31
5.3. Recorrido de un segmento de audio y origen de cada decisión de diseño. La columna derecha indica qué experimento o qué incidente fijó cada etapa; ninguna se eligió por convención.	34
5.4. Izquierda: coste de trocear por reloj, con y sin la duplicación en las fronteras. Derecha: WER frente a latencia media según dónde se corte; las etiquetas indican el tope de duración. Generada desde los datos de ejecución.	35
5.5. Degradación frente al ruido. La banda marca la zona en la que ningún tipo de ruido degrada más de 1,5 puntos, que es el ancho aproximado del intervalo de confianza que corresponde a una muestra de este tamaño según la sección 5.1.2; no se estima de nuevo sobre este barrido. Generada desde los datos de ejecución.	39

Índice de Tablas

3.1. Requisitos funcionales del sistema de subtitulado.	13
3.2. Requisitos no funcionales del sistema de subtitulado.	14
3.3. Requisitos legales y de protección de datos.	15
3.4. Requisitos del procedimiento experimental.	16
3.5. Trazabilidad de requisitos: dónde se resuelve cada uno y con qué evidencia.	17
4.1. Criterios de cumplimiento de los objetivos específicos.	19
5.1. Corpus empleados en la comparativa. Ninguno es aula real; la justificación de cada uno y sus limitaciones se detallan en el capítulo 2.	22
5.2. WER (%) de los modelos Whisper sin adaptar según corpus, con normalización básica. La columna Δ tildes indica cuánto baja el WER de medium al ignorar la acentuación: valores altos delatan referencias con tildes omitidas, no mejor reconocimiento. El símbolo † marca las filas cuyas celdas no comparten tamaño de muestra (teleconciencia-es): esos corpus se ampliaron después del barrido inicial y solo el modelo elegido se reevaluó sobre la ampliación, de modo que comparar columnas dentro de esa fila no es válido. Para comparar modelos entre sí, usar las tablas por corpus, que declaran la muestra fila a fila.	24
5.3. Rendimiento de los modelos Whisper sin adaptar sobre tedx-es (5.5 minutos de audio en la muestra de referencia). Normalización basico . RTF: factor de tiempo real (menor es mejor).	24
5.4. Modelo base de la comparativa frente a una arquitectura de transductor, sobre los mismos clips. Crít. es la tasa de error sobre negaciones, numerales y cuantificadores, y Crít. s/n la misma tasa excluyendo numerales, cuyo recuento mide en parte la convención de escritura: la referencia escribe «dos mil diez» donde Whisper escribe «2010», y el alineamiento contabiliza tres errores por una cifra bien reconocida. Vel. es cuántas veces más rápido que el audio se procesa. Δ negativo indica mejora respecto al modelo base. Las decodificaciones no son equiparables entre arquitecturas: cada sistema se evalúa en su configuración estándar.	26

5.5.	Efecto de cada técnica de adaptación sobre el WER (%), con diseño pareado. Δ negativo indica mejora. El intervalo de confianza se obtiene por bootstrap remuestreando clips; p corresponde al test de signos sobre los clips que cambian. La columna de significación distingue tres situaciones que no deben confundirse: <i>Sí</i> cuando ambos criterios coinciden, <i>En el límite</i> cuando discrepan entre sí, y <i>Sin potencia</i> cuando la muestra no alcanza las 5.000 palabras de referencia que exige la sección 5.1.2. Estas últimas no dicen que la técnica no funcione: dicen que con ese material no se puede saber, y por eso no se interpretan.	27
5.6.	Segmentación por silencios frente a troceado por reloj, con el mismo tope de duración en cada par para que la comparación no mida simplemente la longitud del segmento. A latencia media equivalente (2.68 frente a 2.93 s), la segmentación por silencios reduce el error 7.37 puntos, a costa de que el percentil 95 pase de 3.00 a 5.92 s. Ese contraste se somete al mismo doble criterio que las técnicas de adaptación, sobre los mismos fragmentos: intervalo de confianza al 95 % de [-9.76, -5.04] puntos por remuestreo de clips, y 30 fragmentos que mejoran frente a 2 que empeoran en el test de signos ($p = 2,5e - 07$).	36
5.7.	WER (%) según relación señal-ruido y tipo de ruido añadido. La mayor degradación con murmullo son 21.76 puntos entre 5 y 0 dB, frente a 2.86 puntos como peor salto con ruido blanco: el fallo no es gradual y depende del espectro del ruido, no solo de su energía.	38
6.1.	Verificación de los objetivos frente a los criterios fijados de antemano. . . .	50
B.1.	Riesgos identificados durante la planificación.	65
B.2.	Riesgos detectados durante la ejecución, ausentes de la planificación inicial.	66
C.1.	Requisitos de hardware del equipo servidor, con la medición que respalda cada uno.	68

C.3. Huella de recursos del servicio de reconocimiento, medida sobre AMD Radeon RX 9070 XT con precisión <code>float16</code> y una ventana completa de 30 s de audio, que es el caso peor. El pico de VRAM mayor observado es 1.63 GB, de donde sale el requisito de 4 GB de la tabla C.1: el margen cubre el gestor de ventanas del propio equipo y la fragmentación de memoria.	68
C.2. Requisitos de entorno y periféricos.	69
C.4. Ocupación en disco del despliegue en contenedores sobre AMD, medida con <code>docker system df -v</code> en el equipo de desarrollo.	70
C.5. Sistemas operativos sobre los que puede ejecutarse el servidor.	72
C.6. Imágenes base de las que parten los contenedores del sistema.	73
C.7. Direcciones de acceso al sistema.	78

Resumen

Quien no puede oír la clase, o aún no domina su lengua, depende de que alguien la transcriba. Este trabajo evalúa si las técnicas habituales de adaptación al dominio mejoran el reconocimiento automático del habla en español, y desarrolla una aplicación de subtitulado en vivo desplegable en un centro. Se comparan tres técnicas sobre Whisper con diseño pareado, bootstrap y test de signos: prompting contextual, post-corrección con un modelo de lenguaje local y ajuste fino con LoRA. Solo la última mejora de forma concluyente, 1,23 puntos de error de palabra; la post-corrección empeora. El resultado principal es otro: segmentar el audio por silencios en lugar de por reloj reduce el error 7,4 puntos a igual latencia, bajo el mismo criterio estadístico y seis veces más que la mejor técnica de adaptación. Se muestra además que el error agregado oculta los fallos que importan en accesibilidad, y se proponen métricas que los exponen.

Palabras clave: accesibilidad educativa; reconocimiento automático del habla; adaptación al dominio; subtitulado en tiempo real; evaluación de sistemas ASR.

Abstract

Students who cannot hear the lecture, or do not yet master its language, depend on someone transcribing what is said. This work evaluates whether the usual domain adaptation techniques for automatic speech recognition improve that transcription in Spanish, and develops a live captioning application deployable on a school network. Three techniques are compared on Whisper using a paired design, bootstrap confidence intervals and sign tests: contextual prompting, post-correction with a local language model, and LoRA fine tuning. Only the last one improves conclusively, reducing word error rate by 1.23 points; post-correction makes it worse. The main finding lies elsewhere: segmenting audio at silences rather than at fixed intervals reduces error by 7.4 points at equal latency, six times more than the best adaptation technique. The work also shows that aggregate word error rate hides the failures that matter for accessibility, and proposes complementary metrics that expose them.

Keywords: educational accessibility; automatic speech recognition; domain adaptation; real time captioning; ASR evaluation.

1. Introducción

1.1. Motivación

Un alumno con discapacidad auditiva que asiste a una clase magistral se enfrenta a un problema que el resto no tiene: la información llega por un canal que no puede usar. Las soluciones habituales pasan por un intérprete de lengua de signos o por un profesional que transcriba en directo, y ambas comparten la misma limitación: dependen de que haya una persona disponible, con formación específica, durante toda la sesión. En la práctica eso significa que muchas clases se quedan sin cobertura.

Hay un segundo grupo, mucho más numeroso, para el que el canal tampoco funciona del todo: el alumnado que todavía no domina la lengua en la que se imparte la clase. En el sistema educativo español no universitario había en el curso 2023-2024 más de un millón de alumnos de nacionalidad extranjera, el 12,2% del total (Ministerio de Educación, Formación Profesional y Deportes, 2024); una parte considerable procede de países donde el castellano no es lengua vehicular. Su dificultad no es sensorial sino de velocidad: entienden el vocabulario cuando lo leen y lo pierden cuando lo oyen a la velocidad del habla espontánea, con la fonética de un hablante nativo y sin posibilidad de volver atrás.

Los dos casos difieren en casi todo menos en lo que necesitan, que es exactamente lo mismo: que lo dicho aparezca escrito, en el momento, sin pedírselo a nadie. Y la evidencia de que eso ayuda al segundo grupo no es una suposición razonable: está medida, y se recoge en el capítulo 2. Conviene decir desde ya que este trabajo *no* ha evaluado el sistema con alumnado de ninguno de los dos colectivos; la segunda población se menciona porque amplía el alcance de lo construido, no porque el trabajo aporte evidencia sobre ella.

El reconocimiento automático del habla ha alcanzado en los últimos años una calidad que invita a plantear la alternativa. Los modelos de propósito general funcionan razonablemente bien en español sin necesidad de entrenarlos, y el equipamiento necesario para ejecutarlos ha dejado de ser exclusivo de centros de cálculo. La pregunta ya no es si la transcripción automática es viable, sino si lo es *con la calidad suficiente* para que un alumno pueda seguir una clase sin perder contenido.

Esa distinción es la que motiva este trabajo. Un sistema que acierta el noventa por ciento de las palabras suena aceptable expresado así; pero si el diez por ciento restante incluye los términos técnicos de la asignatura, los nombres propios o las negaciones, el

alumno no recibe una versión ligeramente degradada de la clase: recibe una versión distinta. La métrica habitual del campo, la tasa de error de palabra, no distingue entre perder una preposición y perder un *no*.

1.2. Planteamiento del trabajo

El trabajo tiene dos núcleos que se abordan por separado y se apoyan mutuamente.

El primero es una comparativa empírica: se evalúa si las técnicas de adaptación al dominio más habituales mejoran el reconocimiento en español sobre material que se parece al de un aula. Se estudian tres: aportar al modelo un glosario de la asignatura como contexto, corregir su salida con un modelo de lenguaje, y ajustar sus pesos mediante LoRA. Cada una se mide con el mismo diseño, sobre el mismo corpus y con el mismo criterio estadístico, de modo que las cifras sean comparables entre sí.

El segundo es una aplicación de subtitulado en vivo, construida para desplegarse en la red local de un centro: el docente emite desde su equipo y el alumnado lee desde el suyo, sin que ningún dispositivo del aula necesite capacidad de cálculo. La aplicación no es una demostración del primer núcleo; es un banco de pruebas que plantea preguntas propias, como cada cuánto trocear el audio o dónde cortarlo, que se responden con la misma metodología experimental.

La relación entre ambos resultó menos unidireccional de lo previsto. La investigación proporcionó a la aplicación criterios de diseño medidos en lugar de intuitos; pero la aplicación detectó a su vez un problema que ningún experimento de laboratorio podía ver, porque solo aparece cuando hay un micrófono y un navegador reales de por medio.

1.3. Aportaciones

Conviene adelantar a qué se ha llegado, porque condiciona cómo debe leerse el resto. Cinco resultados resumen el trabajo.

1. **De las tres técnicas de adaptación, solo una mejora.** El ajuste fino con LoRA reduce el error de palabra 1,23 puntos con dos criterios estadísticos de acuerdo. El prompting contextual no produce efecto detectable, y la post-corrección con un modelo de lenguaje *degrada* de forma significativa y replicada sobre dos corpus.
2. **El margen no está donde se suponía.** Segmentar el audio por silencios en lugar

de por reloj reduce el error 7,4 puntos a igual latencia media, con los dos mismos criterios estadísticos de acuerdo: seis veces más que la mejor técnica de adaptación, y a coste de cómputo nulo. La premisa de partida del trabajo queda invertida por sus propios datos.

3. **El WER agregado oculta lo que importa en accesibilidad.** Con un 18,61 % de error, cifra que suena tolerable, el sistema pierde o altera el 28 % de las negaciones e introduce trece que nadie pronunció. Se proponen dos métricas complementarias que exponen esos fallos.
4. **El modo de fallo pesa más que la tasa de error al elegir modelo.** El criterio descartó primero a la variante más rápida de la familia elegida, y después a un candidato de arquitectura distinta que ganaba en las métricas pero traducía al inglés ante habla española difícil.
5. **Varias mediciones aparentemente correctas eran falsas.** Se documentan ocho incidentes en los que el procedimiento producía cifras verosímiles y erróneas sin emitir ningún error, y las comprobaciones que se derivaron de ellos.

En el plano aplicado, el trabajo entrega además un sistema de subtitulado desplegable en la red de un centro, con el reconocimiento centralizado en el servidor, y en el que la técnica ganadora se incorporó sin modificar una sola línea de la aplicación.

1.4. Estructura del trabajo

El capítulo 2 sitúa el trabajo respecto a la literatura y a la práctica del sector, y documenta la búsqueda de corpus, que resultó ser el condicionante principal de todo el diseño experimental. El capítulo 3 deriva los requisitos, tanto funcionales de la aplicación como metodológicos de la comparativa. El capítulo 4 enuncia los objetivos y los criterios con los que se consideran cumplidos.

El capítulo 5 contiene el grueso del trabajo: la metodología experimental, los diez experimentos realizados con sus resultados, y el desarrollo de la aplicación. El capítulo 6 recoge las conclusiones, la verificación de los objetivos, las limitaciones reconocidas y las líneas que quedan abiertas. El apéndice A documenta cómo regenerar cualquier cifra de la memoria, el apéndice B el registro de riesgos que guió la planificación, y el apéndice C los pasos concretos para poner el sistema en marcha en un centro.

Conviene anticipar una particularidad de este trabajo: buena parte de sus aportaciones no proviene de que las técnicas estudiadas funcionaran, sino de descubrir bajo qué condiciones las mediciones que las evalúan son válidas. Varias de las conclusiones surgieron de detectar que un resultado aparentemente sólido era un artefacto del procedimiento de medida. Esos hallazgos se documentan con el mismo detalle que los resultados positivos, porque condicionan cómo deben interpretarse todos los demás.

2. Contexto y Estado del Arte

2.1. El problema: subtítular una clase para quien no puede seguirla

2.1.1. El marco normativo

El derecho a la educación inclusiva de las personas con discapacidad está reconocido en el artículo 24 de la Convención de Naciones Unidas sobre los Derechos de las Personas con Discapacidad (Organización de las Naciones Unidas, 2006), ratificada por España, y desarrollado en la legislación española por el texto refundido de la Ley General de derechos de las personas con discapacidad (Jefatura del Estado, 2013), que obliga a los centros a realizar los ajustes razonables necesarios. En la práctica, el ajuste habitual para el alumnado con discapacidad auditiva en una clase magistral es humano: un intérprete de lengua de signos o un transcriptor en directo.

Existe además una norma que fija qué es un subtítulo aceptable. La UNE 153010 (AENOR, 2012) establece criterios de velocidad de lectura, segmentación y presentación para el subtítulado dirigido a personas sordas. Este trabajo no la aplica como requisito de conformidad, porque el sistema desarrollado no es un producto certificable; sí la toma como referencia de dos decisiones de interfaz: que el texto se presente en unidades sintácticas completas, y que el tamaño lo controle quien lee.

La distancia entre esa norma y lo que un reconocedor automático produce es precisamente el objeto del trabajo. Y hay evidencia previa de que la métrica habitual no mide esa distancia: al evaluar con personas sordas o con discapacidad auditiva la utilidad de subtítulos generados automáticamente, la tasa de error de palabra predice mal la comprensión percibida, porque no todos los errores estorban igual (Kafle y Huenerfauth, 2017). Este trabajo llega a la misma conclusión por una vía distinta, midiendo qué unidades léxicas sobreviven.

2.1.2. Para quién funciona un subtítulo

El colectivo que motiva la norma no es el único que se beneficia, y conviene documentarlo porque amplía sustancialmente el alcance de un sistema como el que aquí se construye. La revisión de más de cien estudios empíricos recogida por Gernsbacher (Gernsbacher,

2015) concluye que el subtitulado en la misma lengua mejora la comprensión, la atención y la memoria del material audiovisual en general, y que resulta *especialmente* útil en tres poblaciones: quienes son sordos o tienen discapacidad auditiva, quienes están aprendiendo a leer, y quienes ven el material en una lengua que no es la suya.

Sobre esa tercera población la evidencia es cuantitativa y antigua. La observación fundacional es de Vanderplank (Vanderplank, 1988), que documentó cómo aprendices de una lengua extranjera explotaban los subtítulos del teletexto para fijar vocabulario que no habrían retenido solo escuchando. El metaanálisis de Montero Perez y colaboradores (Montero Perez, Van Den Noortgate, y Desmet, 2013), sobre dieciocho estudios, encuentra un efecto grande tanto en comprensión oral como en adquisición de vocabulario, con un tamaño del efecto de 0,87 para esta última. El mecanismo que se propone es el que cabría esperar: el subtítulo ancla la cadena sonora a una forma escrita que el aprendiz sí reconoce, y convierte en léxico conocido lo que de otro modo pasaría como ruido.

Ahora bien, esa literatura mide casi siempre *vídeo grabado*, y este trabajo construye un sistema *en vivo*, que no es lo mismo: no hay posibilidad de pausar, ni de rebobinar, ni de ver dos veces, que son precisamente las estrategias que el aprendiz emplea en los estudios anteriores. El único trabajo localizado sobre transcripción en directo en un aula de segunda lengua (Q. Wang y Chen, 2023) obtiene un resultado más matizado, y merece la pena reproducirlo con exactitud: las transcripciones en vivo **no** mejoraron las calificaciones de ningún grupo, pero el alumnado de menor competencia lingüística completó un 24,6 % más de actividades de clase cuando disponía de ellas. Es decir, el efecto medido en directo no aparece en el rendimiento sino en la *participación*: el subtítulo no hace que el alumno aprenda más deprisa, hace que pueda seguir la clase en lugar de descolgarse de ella.

Esa distinción importa para no sobrevender el sistema. Lo que la evidencia sostiene es que un subtítulo en vivo mantiene enganchado a quien de otro modo perdería el hilo; lo que no sostiene todavía, porque no está medido, es que acelere el aprendizaje de la lengua en ese escenario. Encaja además con lo que se observa en el conjunto del alumnado: en una encuesta a 2.124 estudiantes de quince universidades (Linder, 2016), el 98,6 % consideró útiles los subtítulos y tres de cada cuatro los usaban como apoyo al estudio, sin tener ninguna discapacidad ni cursar en lengua extranjera.

La conclusión que este trabajo extrae de esa literatura es de diseño y no de resultados: un sistema de subtitulado de aula tiene una población objetivo bastante mayor que el colectivo que lo justifica normativamente, y por tanto tiene sentido que la vista del alumnado

sea abierta y sin fricción, en lugar de un recurso que haya que solicitar. Es la decisión que se describe en el capítulo 5, y esta sección es su fundamento.

Hay una consecuencia menos cómoda, y también se declara aquí. Si el sistema sirve a quien está aprendiendo la lengua, entonces los modos de fallo estudiados en este trabajo le afectan *más* que al lector nativo, no menos: quien domina el castellano puede reconstruir por contexto una negación perdida o un término mal transcrito, y quien lo está aprendiendo, no. El argumento a favor de medir lo que el WER agregado esconde se refuerza, por tanto, exactamente en la población para la que la evidencia de beneficio es más sólida.

2.2. Reconocimiento automático del habla de propósito general

El punto de partida de este trabajo es Whisper (Radford y cols., 2023), un modelo codificador-decodificador entrenado sobre una cantidad masiva de audio con supervisión débil. Su relevancia para el problema planteado no está en alcanzar el mejor resultado en un banco de pruebas concreto, sino en dos propiedades prácticas: funciona en español sin necesidad de entrenamiento adicional, y es lo bastante robusto frente a condiciones de grabación variables como para plantear su uso fuera de un estudio.

El modelo procesa el audio en ventanas de treinta segundos. Esa decisión de diseño, que resulta natural para transcribir grabaciones completas, se convierte en un obstáculo cuando se pretende subtítular en directo: el sistema no puede esperar medio minuto para mostrar la primera palabra. La adaptación de un modelo pensado para procesamiento por lotes a un escenario de flujo continuo es un problema abierto (Macháček, Dabre, y Bojar, 2023), y constituye una parte sustancial del trabajo de ingeniería descrito en el capítulo 5.

2.2.1. Una familia alternativa: los transductores

Whisper pertenece a la familia codificador-decodificador autorregresiva: genera el texto palabra a palabra condicionando cada predicción en las anteriores. Existe una segunda familia, el transductor neuronal (Graves, 2012), que emite símbolos alineados con las tramas de audio y no dispone de ese bucle sobre el texto ya generado. Es la arquitectura dominante en los sistemas pensados para funcionar en directo, precisamente porque no necesita ver la señal completa para empezar a emitir.

Los sistemas recientes de esta familia combinan un codificador convolucional-atencional

(Gulati y cols., 2020; Rekesh y cols., 2023) con una variante del transductor que predice conjuntamente los símbolos y su duración (Xu y cols., 2023), lo que reduce drásticamente el número de pasos de decodificación y, con ello, el coste por segundo de audio.

Esta sección existe porque el capítulo 5 contrasta el modelo elegido con un sistema de esta segunda familia. La diferencia arquitectónica no es un detalle taxonómico: determina qué se puede controlar del proceso de decodificación, y ese control resultó ser el criterio que decidió la comparación.

2.3. Técnicas de adaptación al dominio

Cuando un modelo general se aplica a un dominio concreto, la terminología específica y el registro propio de ese dominio suelen degradar el resultado. Las estrategias para corregirlo se ordenan por coste creciente.

Condicionamiento por contexto. La más barata consiste en aportar al decodificador un texto previo que oriente sus predicciones, sin modificar el modelo. En Whisper esto se implementa mediante un prompt que actúa como transcripción anterior. Es atractiva porque no requiere entrenamiento ni datos anotados: bastaría con que el docente proporcionara el glosario de la asignatura.

Corrección posterior con un modelo de lenguaje. Una segunda vía deja intacto el reconocedor y repara su salida mediante un modelo de lenguaje, aprovechando que este dispone de conocimiento léxico y sintáctico del que el reconocedor carece. La intuición es sólida; los resultados de este trabajo la contradicen, y el capítulo 5 explica por qué.

Ajuste fino con adaptadores. La tercera modifica los pesos del modelo. El ajuste completo resulta inviable en equipamiento modesto, pero LoRA (Hu y cols., 2021) permite entrenar únicamente unas matrices de rango bajo insertadas en las capas de atención, reduciendo los parámetros entrenables en varios órdenes de magnitud y produciendo adaptadores de unos pocos megabytes. Es lo que hace que esta técnica sea abordable en el contexto de un trabajo como este.

2.4. Recursos de habla en español

La disponibilidad de corpus condicionó el diseño experimental más que ninguna otra decisión, hasta el punto de que conviene documentar la búsqueda y no solo su resultado.

2.4.1. El corpus objetivo

El recurso que mejor se ajusta al problema es poliMedia, el repositorio de clases grabadas de la Universitat Politècnica de València, del que el proyecto europeo transLectures transcribió manualmente más de ciento quince horas (del Agua y cols., 2014). Reúne las tres condiciones que este trabajo necesita: dominio educativo real, lengua española y transcripción verificada por personas.

No se encuentra publicado en ningún repositorio de acceso abierto, y su sitio web rechazó la conexión de forma repetida durante la fase de auditoría de corpus, de modo que no fue posible confirmar por esa vía ni el procedimiento de acceso ni la licencia. El repositorio público del grupo distribuye otros conjuntos de datos, todos en inglés, pero no este: el único camino es la solicitud directa, que queda redactada y dirigida al grupo responsable como continuación inmediata del trabajo. Esta circunstancia es la limitación de fondo de toda la parte experimental.

2.4.2. Los recursos efectivamente empleados

A falta del corpus objetivo se recurrió a sustitutos, escogidos para cubrir ejes distintos del problema en lugar de acumular horas.

VoxPopuli (C. Wang y cols., 2021) aporta discurso parlamentario en español peninsular. Presenta dos propiedades que resultaron decisivas: incorpora una marca que identifica las transcripciones verificadas manualmente, y anota la variedad dialectal del hablante. Es, por eso, el único de los cinco sobre el que se realiza el ajuste fino. FLEURS (Conneau y cols., 2022) proporciona voz leída de dominio general, útil como techo de calidad; conviene señalar que su configuración en español corresponde a la variedad latinoamericana, sin equivalente peninsular.

Los dos recursos de habla espontánea proceden del proyecto CIEMPIESS de la Universidad Nacional Autónoma de México (Hernández Mena y Herrera Camacho, 2014): un conjunto de charlas expositivas y otro de divulgación científica televisiva. Son los que más se acercan al registro de una clase magistral entre los disponibles sin trámites, y también los de peor calidad de referencia, como se explica a continuación. Se añadió por último un conjunto de locución de medios en español peninsular (Kolobov y cols., 2021), que cubre el eje dialectal a registro controlado.

Se descartó Common Voice (Ardila y cols., 2020) pese a su volumen: son frases leídas y

aisladas, sin la continuidad discursiva que caracteriza a una explicación de aula, de modo que habría aportado horas sin aportar el fenómeno que se quería medir.

2.4.3. La calidad de la referencia como variable

Un hallazgo de la fase de búsqueda merece mención aquí porque afecta a la interpretación de toda la literatura basada en estos recursos: la calidad de las transcripciones de referencia varía sustancialmente entre corpus abiertos, y esa variación se confunde con diferencias de rendimiento del modelo.

En los recursos de habla espontánea empleados, las referencias omiten la acentuación de forma sistemática y contienen erratas. El efecto se cuantifica reevaluando el mismo resultado con un normalizador que ignora las tildes: la columna Δ *tildes* de la tabla 5.2 recoge esa diferencia. Sobre los corpus mexicanos alcanza 3,48 y 5,11 puntos, es decir, entre un cuarto y un tercio de todo el error atribuido al modelo; sobre los peninsulares, con referencia verificada, cae a 0,07 puntos. La brecha no mide reconocimiento: mide la referencia.

Se observaron además casos en que el sistema escribía correctamente una palabra que la transcripción de referencia recogía mal, y era penalizado por ello: *mallorca* donde la referencia decía *mayorca*, *velázquez* donde decía *velásquez*.

La consecuencia práctica quedó incorporada al procedimiento: antes de adoptar un corpus se audita manualmente una muestra de sus referencias. Que un recurso disponga de transcripción no implica que sirva como referencia.

2.5. Evaluación de sistemas de reconocimiento

2.5.1. La métrica habitual y sus límites

La tasa de error de palabra, definida como la distancia de edición entre referencia e hipótesis normalizada por la longitud de la referencia, es la métrica estándar del campo. Su fortaleza es la comparabilidad; su debilidad, que asigna el mismo coste a todas las palabras y a todas las operaciones de edición.

Esa limitación está documentada en la literatura, que señala que la métrica opera en el plano léxico sin consideración del contexto semántico, y que trata de forma idéntica errores que destruyen la comprensión y errores irrelevantes para ella. Se han propuesto alternativas que combinan la distancia de edición con señales de distancia semántica entre

referencia e hipótesis (Kim y cols., 2021), y en el ámbito concreto del subtítulo accesible se ha mostrado experimentalmente, con usuarios sordos o con discapacidad auditiva, que dos transcripciones con el mismo WER pueden diferir mucho en cuánto se entiende de ellas (Kafle y Huenerfauth, 2017).

Conviene precisar también cómo se calcula aquí el intervalo de confianza de esa métrica. El remuestreo por bootstrap (Efron y Tibshirani, 1993) es el procedimiento habitual para ello en reconocimiento del habla, con una advertencia recogida en la literatura del campo desde hace dos décadas (Bisani y Ney, 2004): la unidad de remuestreo debe ser el fragmento y no la palabra, porque los errores se agrupan dentro de una misma locución y tratarlos como independientes estrecha el intervalo de forma artificial.

Este trabajo llega a la misma conclusión por vía empírica y desde un dominio concreto: los errores que perjudican a un alumno con discapacidad auditiva no son los que más pesan en la métrica. Las mediciones que lo sustentan, y los instrumentos complementarios que se proponen, se presentan en el capítulo 5.

2.5.2. Alucinaciones

Un modo de fallo específico de los modelos generativos aplicados al reconocimiento consiste en producir texto fluido y verosímil sin correspondencia con el audio. A diferencia de un error ordinario, resulta indetectable para el usuario final: la salida es gramatical y coherente con el dominio.

Para un sistema de accesibilidad esta distinción es central. Un subtítulo ausente se percibe como una carencia; uno inventado se lee como información. El trabajo incorpora por ello la detección de salidas anómalas como criterio de selección de modelo, y no solo la tasa de error.

2.6. Subtitulado en tiempo real

La literatura sobre adaptación de Whisper a escenarios de flujo continuo (Macháček y cols., 2023) y la práctica documentada del sector coinciden en varios puntos que sirvieron de referencia para el diseño de la aplicación: emplear ventanas de entre cinco y diez segundos, solaparlas para no perder las palabras que caen en la frontera, y apoyarse en detección de actividad de voz para decidir cuándo dar por terminada una intervención. Para esto último la práctica habitual recurre a detectores entrenados (Siler Team, 2021); este trabajo

emplea uno de energía, y la sección 6.3 razona por qué esa elección no invalida el hallazgo que sostiene.

Coinciden también en desaconsejar el condicionamiento sobre la transcripción previa entre ventanas, por el riesgo de propagación de errores. Este trabajo verificó esa recomendación de forma independiente y obtuvo el mismo resultado, lo que constituye una validación externa útil de la parte experimental.

La divergencia principal respecto a la práctica habitual está en el umbral de silencio empleado para cerrar una intervención, y se discute en el capítulo 5 junto con la medición que la respalda.

2.7. Posicionamiento del trabajo

Este trabajo no propone una técnica nueva de adaptación ni un modelo nuevo. Su aportación se sitúa en tres planos.

En el plano empírico, compara tres técnicas de adaptación sobre español con un diseño y un criterio estadístico homogéneos, y contrasta su rendimiento con el de las decisiones de ingeniería del sistema; una comparación que la literatura suele tratar por separado.

En el plano metodológico, documenta las condiciones bajo las cuales esas mediciones son válidas, incluyendo varios casos en que un procedimiento aparentemente correcto produce cifras verosímiles y falsas.

En el plano aplicado, construye un sistema desplegable en un centro educativo y lo emplea como banco de pruebas, del que surgieron preguntas que la parte experimental no se habría planteado.

3. Identificación de Requisitos

El trabajo tiene dos núcleos y por tanto dos conjuntos de requisitos de naturaleza distinta: los que debe cumplir la aplicación para ser útil en un aula, y los que debe cumplir el procedimiento experimental para que sus conclusiones sean válidas. Los segundos suelen quedar implícitos; aquí se enuncian porque varias decisiones del trabajo se derivan directamente de ellos.

3.1. Requisitos del sistema de subtitulado

3.1.1. Funcionales

Los requisitos funcionales, recogidos en la tabla 3.1, describen lo que el sistema debe hacer desde el punto de vista de quien lo usa: el docente que emite y el alumnado que lee.

Id	Requisito
RF1	El sistema transcribe en directo la voz del docente y muestra el resultado como subtítulos.
RF2	Varios alumnos pueden visualizar simultáneamente los subtítulos desde sus propios dispositivos.
RF3	El alumnado no necesita instalar nada ni conceder permisos: basta con abrir una dirección en el navegador.
RF4	El docente puede definir, antes de comenzar, la terminología de la sesión, que se resalta visualmente en los subtítulos.
RF5	Un alumno que se incorpora una vez iniciada la clase recupera lo transcrito hasta ese momento.
RF6	Cada alumno ajusta el tamaño del texto en su propio dispositivo.
RF7	El sistema informa de la latencia con que está operando.

Tabla 3.1: Requisitos funcionales del sistema de subtitulado.

El requisito RF6 no es un detalle de comodidad. El sistema se dirige a personas con una discapacidad sensorial, y no cabe suponer que quien lo desarrolla pueda fijar un tamaño de texto adecuado para todas ellas ni para todas las distancias de lectura de un aula.

RF3 tiene una justificación menos evidente y más importante de lo que aparenta. La población que se beneficia de un subtítulo de aula es mayor que el colectivo que lo motiva: incluye al alumnado que aún no domina la lengua de la clase, con evidencia empírica detrás

(sección 2.1.2). Un recurso que hubiera que solicitar, instalar o activar excluiría por diseño a casi toda esa población, y obligaría a identificarse a quien prefiriera no hacerlo. De ahí que el requisito no sea «que sea fácil de usar» sino que no haya nada que hacer: abrir una dirección.

3.1.2. No funcionales

Los de la tabla 3.2 no describen funciones sino condiciones bajo las cuales las anteriores son útiles: de poco sirve transcribir bien si el subtítulo llega tarde, si el sistema necesita una unidad de cálculo por alumno o si el audio del aula sale del centro.

Id	Requisito
RNF1	La latencia entre lo dicho y lo mostrado debe permitir seguir la clase. El umbral concreto se determina experimentalmente y se reporta con su valor típico y su peor caso.
RNF2	El coste de cálculo no debe crecer con el número de alumnos conectados.
RNF3	El sistema opera exclusivamente en la red local del centro.
RNF4	El puesto de emisión está protegido frente a uso no autorizado.
RNF5	El sistema funciona con equipamiento asequible para un centro educativo.
RNF6	El requisito de captación de audio debe expresarse en términos verificables, no como recomendación genérica.

Tabla 3.2: Requisitos no funcionales del sistema de subtítulo.

RNF2 es el que determina la arquitectura. Si cada dispositivo del aula ejecutara el modelo, el sistema exigiría una unidad de cálculo por alumno y dejaría de ser desplegable. De ahí que todo el reconocimiento resida en el servidor y los clientes se limiten a visualizar.

RNF3 se deriva de la naturaleza del material: el sistema difunde audio de aula transcrito, con voces identificables y frecuentemente de menores de edad. La restricción no puede delegarse en la configuración de red del centro, porque un reenvío de puertos mal configurado la anularía sin que nadie lo advirtiera; debe imponerla la propia aplicación.

RNF6 responde a un problema recurrente en los requisitos de este tipo de sistemas. Indicar que se necesita “un micrófono adecuado” no es verificable. La caracterización experimental frente al ruido permite sustituirlo por una condición comprobable sobre la relación señal-ruido, y con ella justificar la elección del equipo de captación.

3.1.3. Legales y de protección de datos

Un sistema que capta la voz de un aula y la difunde por una red trata datos personales, y frecuentemente de menores. Los requisitos que se derivan de ello no son una formalidad añadida al final: dos de los recogidos en la tabla 3.3 determinaron decisiones de arquitectura.

Id	Requisito
RL1	La voz captada no abandona la red del centro ni se envía a ningún servicio de terceros. El reconocimiento se ejecuta en un equipo del propio centro.
RL2	El sistema no almacena el audio. La transcripción vive en memoria mientras dura la sesión y se descarta al terminarla.
RL3	La restricción de alcance la impone la aplicación, no la configuración de red del centro, de modo que sea verificable y no dependa de una suposición.
RL4	El material empleado en la comparativa procede de corpus con licencia de uso para investigación; no se grabaron clases reales, y por tanto no se recabaron consentimientos ni se trataron datos de alumnado identificable.

Tabla 3.3: Requisitos legales y de protección de datos.

RL1 es la razón de que el reconocimiento se ejecute en local en lugar de delegarse en un servicio comercial en la nube, que habría sido más sencillo y probablemente más preciso. Enviar audio de aula a un tercero exigiría una base jurídica y un contrato de encargo de tratamiento (Parlamento Europeo y Consejo de la Unión Europea, 2016) que quedan fuera de lo que un centro puede resolver por su cuenta para una asignatura. Ejecutarlo en local convierte un problema de cumplimiento en un problema de ingeniería, que es la razón de fondo de la restricción de recursos que atraviesa todo el trabajo.

RL4 delimita el alcance ético de la parte experimental: al no grabarse material propio, no hubo tratamiento de datos personales que requiriera consentimiento informado ni evaluación ética. La continuación prevista en el capítulo 6 es repetir la comparativa sobre clases reales; si llega a ejecutarse, ese punto deja de ser trivial y debe resolverse antes de grabar, no después.

3.2. Requisitos del procedimiento experimental

Estos requisitos, listados en la tabla 3.4, no proceden del usuario sino de la naturaleza de la pregunta que el trabajo pretende responder. Se enuncian porque su incumplimiento no produce un fallo visible: produce cifras verosímiles y falsas.

Id	Requisito
RE1	Toda cifra publicada debe poder regenerarse a partir del código y los datos registrados junto a ella.
RE2	La identidad de un resultado incluye su configuración completa, no solo el modelo y el corpus.
RE3	El tamaño de muestra debe justificarse antes de interpretar diferencias, y las mediciones que no lo alcancen deben marcarse como no concluyentes.
RE4	Las comparaciones entre condiciones deben ser pareadas y compartir configuración.
RE5	Los datos empleados para adaptar el sistema deben ser disjuntos de los empleados para evaluarlo.
RE6	La calidad de las referencias debe auditarse antes de adoptarlas.
RE7	Ninguna cifra de la memoria se transcribe manualmente: las tablas y figuras se generan desde los datos de ejecución.

Tabla 3.4: Requisitos del procedimiento experimental.

RE2 merece justificación porque resulta menos evidente que los demás. Durante el trabajo se comprobó que dos ejecuciones del mismo modelo sobre el mismo corpus, diferenciadas únicamente por la estrategia de decodificación, arrojaban tasas de error que diferían en más de doce puntos. Un resultado identificado solo por modelo y corpus habría permitido comparar cifras no comparables sin que nada lo delatara.

RE6 se incorporó tras comprobar que una parte sustancial del error atribuido al modelo en determinados corpus correspondía en realidad a defectos de la transcripción de referencia.

RE7 es una consecuencia práctica de RE1: una cifra copiada a mano deja de estar vinculada a la ejecución que la produjo en el momento en que se copia.

3.3. Trazabilidad de los requisitos

Enunciar requisitos sin cerrarlos deja al lector sin forma de comprobar si se cumplieron. La tabla 3.5 los vincula con el mecanismo concreto que los satisface y con la evidencia que lo demuestra; el capítulo 5 desarrolla cada uno y el capítulo 6 verifica los objetivos que de ellos se derivan.

Id	Dónde se resuelve	Cómo se comprueba
RF1	Segmentador en vivo y servicio de reconocimiento	Demostración funcional; sesión con micrófono real
RF2	Sesión de subtítulo compartido, difusión por websocket	Pruebas automáticas de la sesión; varios clientes simultáneos
RF3	Vista de alumnado sin captura ni permisos	Abrir la dirección en un navegador sin instalar nada
RF4	Decorador de resaltado sobre el motor de reconocimiento	Pruebas del glosario, incluida la protección de la eñe
RF5	Histórico retenido en la sesión y enviado al conectar	Prueba automática de incorporación tardía
RF6	Control de tamaño de texto en la vista del alumnado	Verificación manual en la interfaz
RF7	Instrumentación de latencia de extremo a extremo en el navegador	Panel de la vista de emisión (sección 5.6.9)
RNF1	Segmentación por pausas con tope de duración	exp-100 y exp-102; medición del circuito completo
RNF2	Reconocimiento único en el servidor para toda el aula	Arquitectura; coste independiente del número de clientes
RNF3	Filtro de red privada en la propia aplicación	Pruebas automáticas del filtro
RNF4	Clave compartida en el puesto de emisión	Pruebas automáticas de acceso
RNF5	Modelo de tamaño medio sobre una GPU de consumo	exp-000 y exp-101
RNF6	Curva de degradación frente al ruido	exp-104: requisito expresado en dB
RL1-RL3	Ejecución local, sin persistencia de audio, filtro de red privada	Pruebas automáticas y revisión de la configuración de despliegue
RE1-RE7	Registro de procedencia y generación automática de tablas y figuras	Apéndice A

Tabla 3.5: Trazabilidad de requisitos: dónde se resuelve cada uno y con qué evidencia.

4. Objetivos

4.1. Objetivo general

Determinar si las técnicas de adaptación al dominio mejoran el reconocimiento automático del habla en español para su uso en accesibilidad educativa, y construir un sistema de subtitulado en vivo que permita comprobarlo en condiciones de aula.

El objetivo se formula deliberadamente como una pregunta abierta y no como la búsqueda de una mejora: un resultado negativo bien medido responde igual de bien que uno positivo, siempre que el procedimiento permita distinguir la ausencia de efecto de la incapacidad para detectarlo.

4.2. Objetivos específicos

Del núcleo investigador

1. Establecer una línea base reproducible del reconocimiento sin adaptar, sobre varios modelos y corpus, con trazabilidad completa de la configuración que produjo cada cifra.
2. Evaluar tres técnicas de adaptación (prompting contextual, post-corrección con un modelo de lenguaje local y ajuste fino con LoRA) bajo un diseño común que permita compararlas entre sí.
3. Determinar la potencia estadística necesaria para que la comparación sea capaz de detectar los efectos esperables, y verificar que el material empleado la alcanza.
4. Identificar qué modos de fallo relevantes para accesibilidad no quedan reflejados en la métrica habitual, y proponer instrumentos que los expongan.

Del núcleo aplicado

5. Construir una aplicación de subtitulado en vivo desplegable en la red local de un centro, con un emisor y múltiples receptores, en la que todo el cálculo resida en el servidor.

6. Caracterizar experimentalmente el compromiso entre latencia y calidad que impone el troceado del audio, y elegir el punto de operación con datos propios.
7. Cuantificar el margen acústico que requiere el sistema, para poder formular requisitos de despliegue verificables.
8. Mantener una frontera entre investigación y aplicación que permita incorporar la técnica ganadora al sistema desplegado sin modificarlo.

4.3. Criterios de cumplimiento

Cada objetivo se considera alcanzado bajo condiciones explícitas, fijadas antes de ejecutar los experimentos para evitar ajustarlas a los resultados obtenidos. La tabla 4.1 los recoge:

Obj.	Se considera cumplido si
1	Cualquier cifra publicada puede regenerarse con un solo comando, y el resultado registra el commit y el contenido exacto del corpus empleado.
2	Las tres técnicas se miden sobre el mismo corpus, con la misma decodificación y el mismo criterio de significación.
3	El tamaño de muestra se justifica con un cálculo previo, y las mediciones que no lo alcanzan se marcan como no concluyentes en lugar de interpretarse.
4	Se dispone de al menos una métrica adicional que detecte fallos invisibles al WER, con evidencia de que tales fallos existen en el material analizado.
5	Un cliente conectado a la red del centro visualiza los subtítulos sin ejecutar ningún cálculo ni requerir permisos de captura.
6	La elección del tamaño y la estrategia de segmentación se respalda con un barrido propio, no con valores tomados por convención.
7	Existe una curva de degradación frente a ruido que permita expresar el requisito de captación en términos verificables.
8	La técnica ganadora se incorpora al sistema desplegado sin modificar el código de la aplicación.

Tabla 4.1: Criterios de cumplimiento de los objetivos específicos.

4.4. Alcance y exclusiones

Quedan fuera del alcance, y se declaran aquí para evitar que se interpreten como carencias: la evaluación con usuarios reales del sistema desplegado; la traducción a otros idiomas o a lengua de signos; el reconocimiento de múltiples hablantes simultáneos; y la identificación de quién habla en cada momento. El sistema asume un único emisor, que es el escenario de la clase magistral.

Queda fuera también la medición del efecto del sistema sobre el alumnado que está aprendiendo el castellano. Ese colectivo se identifica en el capítulo 2 como segunda población beneficiaria, con la evidencia que lo respalda, y condiciona una decisión de diseño concreta: que la vista de recepción sea abierta. Pero ningún objetivo de este trabajo se formula sobre él, porque comprobarlo exige evaluación con personas.

5. Desarrollo del trabajo

5.1. Metodología experimental

5.1.1. Principios de diseño

Toda la comparativa descansa sobre cuatro decisiones metodológicas que se fijaron antes de ejecutar ningún experimento.

Diseño pareado. Cada técnica se evalúa sobre exactamente los mismos fragmentos de audio que su línea base, en la misma ejecución y con la misma configuración de decodificación. Esto permite comparar fragmento a fragmento en lugar de comparar dos medias, que es lo único que proporciona algo de potencia estadística con corpus de tamaño moderado.

Doble criterio de significación. Se calcula un intervalo de confianza al 95 % para la diferencia de error mediante remuestreo por bootstrap, y un test de signos sobre los fragmentos que cambian. Una diferencia se considera significativa solo si ambos coinciden. El motivo es práctico: durante el trabajo apareció un caso en que el intervalo excluía el cero por muy poco y el test de signos no lo respaldaba; interpretarlo como efecto real habría sido forzar el dato.

Remuestreo por fragmentos, no por palabras. Los errores de reconocimiento se agrupan dentro de un mismo fragmento y un mismo hablante. Tratar las palabras como observaciones independientes estrecha el intervalo de confianza de forma artificial y produce significación donde no la hay.

Trazabilidad. Cada resultado registra el commit del código, si el árbol de trabajo estaba limpio, y el resumen criptográfico del manifiesto de corpus empleado. La razón se explica en la sección 5.7.

Conviene precisar hasta dónde llega, porque el principio se adoptó *durante* el trabajo y no antes: el resumen del manifiesto se escribió como respuesta al incidente que describe la sección 5.7, de modo que cubre once de los cincuenta y dos resultados, los posteriores. Todos registran configuración y commit, pero en cuarenta y cuatro ese commit vale «sin repositorio», porque son anteriores a que el trabajo estuviera versionado. Para esos, la garantía de que el corpus no ha cambiado por debajo se sustituye por una comprobación indirecta y automatizada que contrasta el número de fragmentos y de palabras declarados contra el manifiesto actual. El apéndice A da el desglose y explica qué señala hoy esa

comprobación.

5.1.2. Potencia estadística requerida

Antes de interpretar ninguna diferencia hubo que responder cuánto material hace falta para poder detectarla. Con 499 palabras de referencia, el error estándar del WER en torno al 3 % ronda 0,8 puntos, de modo que el intervalo de confianza alcanza aproximadamente $\pm 1,5$ puntos: bajo esas condiciones, tres modelos que difieren en 0,4 puntos son indistinguibles.

Para detectar diferencias del orden de 0,5 puntos, que es la escala en que se mueven las técnicas de adaptación, hacen falta del orden de 30.000 palabras de referencia; unas tres o cuatro horas de audio transcrito. La cifra es una cota optimista, porque supone errores independientes cuando en realidad se agrupan.

Esta estimación condicionó todo lo demás: las mediciones que no la alcanzan se marcan como no concluyentes y no se interpretan, ni a favor ni en contra.

5.1.3. Corpus

El obstáculo principal del trabajo fue localizar audio educativo en español con transcripción de referencia fiable. La búsqueda y auditoría de candidatos se documenta en el capítulo 2; aquí importa el resultado: se emplearon cinco corpus de características deliberadamente distintas, resumidos en la tabla 5.1.

Corpus	Variedad	Registro	Clips
fleurs-es	Latinoamérica	Voz leída, dominio general	20
tedx-es	México	Charla expositiva espontánea	40
teleconciencia-es	México	Divulgación científica en televisión	1.200
mediaspeech-es	Peninsular	Locución de medios	40
voxpopuli-es	Peninsular	Debate parlamentario	400

Tabla 5.1: Corpus empleados en la comparativa. Ninguno es aula real; la justificación de cada uno y sus limitaciones se detallan en el capítulo 2.

Ninguno de ellos es una clase grabada. Esta es la limitación de fondo del trabajo y condiciona la validez externa de todas las conclusiones; se discute en la sección 6.3.

5.2. Línea base sin adaptación

5.2.1. Elección del modelo

Se evaluaron seis tamaños de Whisper sobre el mismo material. La figura 5.1 muestra el compromiso entre calidad y coste, la tabla 5.2 recoge el resumen por corpus y la tabla 5.3 el detalle sobre habla espontánea, que es el registro donde la elección se juega.

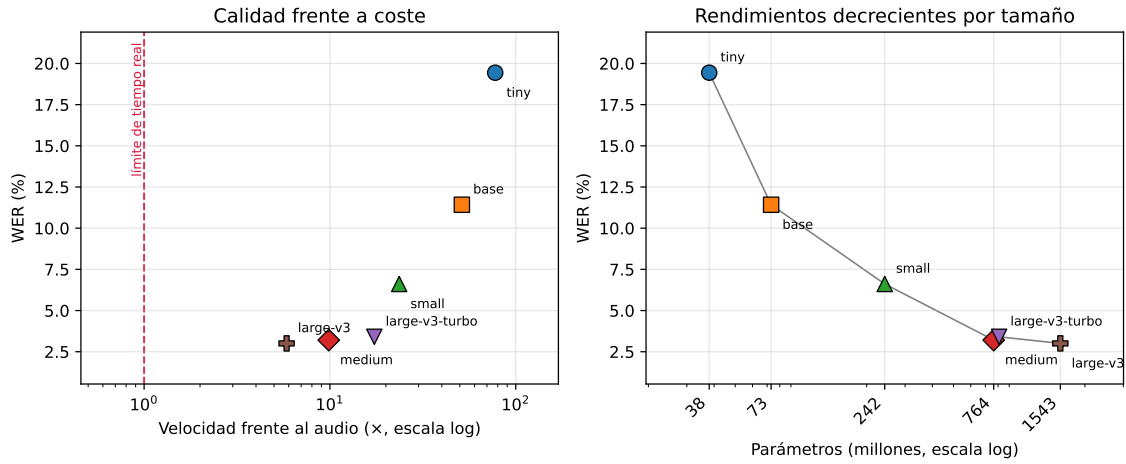


Figura 5.1: Calidad frente a coste y rendimientos decrecientes por tamaño de modelo. La línea vertical marca el límite de tiempo real. Generada desde los datos de ejecución.

Antes de leer la tabla conviene una advertencia sobre ella. Sus columnas no comparten en todos los casos el tamaño de muestra, porque un corpus se amplió después del barrido inicial y el modelo elegido se reevaluó sobre la ampliación. Las comparaciones *entre modelos* deben hacerse por tanto sobre las tablas por corpus, que declaran su muestra fila a fila; la tabla agregada sirve para comparar *material*, que es para lo que se construyó. El origen de esa asimetría se documenta en la sección 5.7.

Hechas esas cuentas sobre muestra común, dos observaciones condicionan el resto del trabajo.

La primera: en material de habla espontánea, aumentar el tamaño del modelo deja de compensar. Entre `medium` y `large-v3-turbo` la diferencia no solo es nula sino que cambia de signo según el corpus, y sobre habla espontánea el modelo mayor es *peor* (tabla 5.3). Entre `small` y `medium` la distancia se estrecha a medida que el material se aleja de la voz leída: 3,40 puntos en FLEURS, 1,91 en charla espontánea y 1,22 en locución de medios.

Corpus	Variedad	Registro	tiny	base	small	medium	large-v3-turbo	large-v3	Δ tildes
fleurs-es	Latinoamérica	Leída, enciclopédica	19.44	11.42	6.61	3.21	3.41	3.01	–
tedx-es	México	Charla espontánea	–	–	12.13	10.22	11.12	–	–3.48
teleconciencia-es [†]	México	Divulgación científica	–	–	23.42	18.61	22.26	–	–5.11
mediaspeech-es	Peninsular	Medios, locución	–	–	15.94	14.72	15.60	–	–1.70
voxpath-es	Peninsular	Parlamentario	–	–	12.01	10.44	11.53	–	–0.07
voxpath-es-400	Peninsular	Parlamentario	–	–	–	9.63	–	–	–0.17

Tabla 5.2: WER (%) de los modelos Whisper sin adaptar según corpus, con normalización básica. La columna Δ tildes indica cuánto baja el WER de **medium** al ignorar la acentuación: valores altos delatan referencias con tildes omitidas, no mejor reconocimiento. El símbolo [†] marca las filas cuyas celdas **no** comparten tamaño de muestra (**teleconciencia-es**): esos corpus se ampliaron después del barrido inicial y solo el modelo elegido se reevaluó sobre la ampliación, de modo que comparar columnas dentro de esa fila no es válido. Para comparar modelos entre sí, usar las tablas por corpus, que declaran la muestra fila a fila.

Modelo	Par. (M)	Clips	WER (%)	CER (%)	RTF	$\times$ tiempo real
whisper-small	242	40	12.13	5.37	0.055	18.2
whisper-medium	764	40	10.22	4.39	0.136	7.4
whisper-large-v3-turbo	809	40	11.12	4.98	0.081	12.3

Tabla 5.3: Rendimiento de los modelos Whisper sin adaptar sobre **tedx-es** (5.5 minutos de audio en la muestra de referencia). Normalización **basico**. RTF: factor de tiempo real (menor es mejor).

El margen que queda no está en el tamaño; ese es precisamente el supuesto que motiva estudiar la adaptación.

La segunda: lo que domina el error no es la variedad dialectal sino el registro. El recorrido del 3% al 23% se explica por lo que se está haciendo con la voz, leer un texto frente a hablar de forma espontánea, y no por de dónde es quien habla: los dos corpus de habla pública preparada, el parlamentario peninsular y la charla mexicana, quedan en 10,44% y 10,22% pese a estar en extremos opuestos del dominio dialectal, mientras que dos corpus de la misma variedad se separan más de diez puntos según el registro.

Conviene no pedirle a esa comparación más de lo que puede dar. Ambas cifras proceden de cuarenta fragmentos, muy por debajo del umbral de la sección 5.1.2, de modo que una diferencia de 0,22 puntos entre ellas no es distinguible de cero y tampoco lo sería una

bastante mayor; y los dos registros son parecidos, no idénticos. Lo que el dato sostiene es que la variedad no aparece como el factor dominante frente al registro, cuyo efecto es de otro orden de magnitud. Lo que no sostiene es que la variedad sea irrelevante. Por eso no se la descarta como factor: se la deja fuera de la comparativa por no poder controlarla con el material disponible, que es una limitación y así consta en la sección 6.3. Explica además por qué ninguno de los sustitutos reemplaza al corpus objetivo, cuyo registro es justamente el que falta.

5.2.2. El modelo elegido y por qué no es el más rápido

La primera lectura recomendaba **large-v3-turbo**: igualaba a **medium** en tasa de error y era el doble de rápido. Esa lectura ignoraba el modo de fallo.

Contabilizando las salidas anómalas, entendidas como transcripciones con menos de un cuarto de las palabras esperadas, **turbo** produjo tres y **medium** ninguna sobre los mismos 180 fragmentos, que es la muestra que ambos comparten. Conviene la precisión, porque **medium** tampoco está libre de ellas: sobre otro material sí las produce, dos en los cuatrocientos fragmentos del corpus parlamentario. Lo que la comparación sostiene es que en igualdad de condiciones aparecen en uno y no en el otro, no que el modelo elegido no alucine. Y no son truncamientos inocuos: en varios fragmentos el modelo emitió una frase fluida, gramatical y plausible en el dominio que no guardaba relación con lo dicho.

Para un sistema de accesibilidad ese criterio pesa más que unas décimas de tasa de error. Un subtítulo ausente se nota; uno inventado, no: el alumno con discapacidad auditiva no tiene forma de detectarlo porque se lee perfectamente bien. Se eligió **medium**.

5.3. Contraste con una arquitectura distinta

El barrido anterior compara variantes de una misma familia, de modo que justifica cuál de ellas emplear pero no la familia en sí. Durante el desarrollo aparecieron modelos multilingües contruidos sobre una arquitectura distinta, con cifras publicadas mejores, y la pregunta pasó a ser exigible: la respuesta a por qué este modelo y no otro no puede ser que fuera el primero disponible.

Se contrastó el modelo elegido con un transductor (sección 2.2.1) de 0,6 mil millones de parámetros sobre los mismos ochocientos clips: cuatrocientos de habla parlamentaria leída y cuatrocientos de habla espontánea, estos últimos una submuestra del corpus grande con

manifiesto propio, de modo que las cifras de esta sección no son directamente comparables con las medidas sobre sus mil doscientos fragmentos. Un transductor emite un símbolo por trama y no dispone del bucle autorregresivo sobre el texto donde el primero aplica su reintento por temperatura, así que no existe una decodificación común: cada sistema se evaluó en la suya, y la comparación es entre sistemas completos.

Corpus	Modelo	Par. (M)	WER	Crít.	Crít. s/n	Vel.	Δ (pp)	IC 95 %	Veredicto
teleconciencia-es-400	whisper-medium	764	19.55	22.17	23.08	8×	—	—	referencia
teleconciencia-es-400	parakeet-tdt-0.6b-v3	627	19.83	10.36	10.00	106×	+0.28	[-1.28, +2.03]	sin evidencia
voxpathli-es-400	whisper-medium	764	9.63	17.52	7.74	8×	—	—	referencia
voxpathli-es-400	parakeet-tdt-0.6b-v3	627	6.83	4.46	4.33	44×	-2.80	[-3.48, -2.20]	mejora

Tabla 5.4: Modelo base de la comparativa frente a una arquitectura de transductor, sobre los mismos clips. **Crít.** es la tasa de error sobre negaciones, numerales y cuantificadores, y **Crít. s/n** la misma tasa excluyendo numerales, cuyo recuento mide en parte la convención de escritura: la referencia escribe «dos mil diez» donde Whisper escribe «2010», y el alineamiento contabiliza tres errores por una cifra bien reconocida. **Vel.** es cuántas veces más rápido que el audio se procesa. Δ negativo indica mejora respecto al modelo base. Las decodificaciones no son equiparables entre arquitecturas: cada sistema se evalúa en su configuración estándar.

La ventaja en tasa de error del transductor resultó ser real pero limitada al habla leída, y menor de lo que aparentaba. Las referencias escriben los números en letra mientras que el modelo empleado los escribe en cifra, y como *dos mil diez* son tres unidades léxicas frente a la única de *2010*, el alineamiento contabiliza tres errores por cada cifra correctamente reconocida. Repitiendo el contraste sobre los clips cuya referencia no contiene numerales, la diferencia baja de 2,80 a 2,31 puntos: sigue siendo significativa, pero cerca de una quinta parte de lo medido era convención de escritura. Sobre habla espontánea no hay diferencia detectable en ningún caso.

El contenido crítico cuenta otra historia. Con tasas de error indistinguibles sobre habla espontánea, el modelo empleado pierde 48 de 138 negaciones y el transductor 16. Es el mismo fenómeno que motivó las métricas complementarias, observado ahora entre arquitecturas: dos sistemas que el WER declara equivalentes difieren en un factor de tres en aquello que determina si la frase conserva su significado.

El criterio que resolvió fue de nuevo el modo de fallo. El transductor no trunca: traduce. Produjo frases completas en inglés ante habla espontánea española y alternancias de idioma

dentro de una misma oración, en trece clips de los ochocientos, frente a ninguno del modelo empleado. Y no es corregible por configuración, porque esa arquitectura no expone ningún mecanismo para imponer el idioma de salida, mientras que al modelo empleado sí se le fija. Se mantuvo por tanto la elección inicial, ahora con una razón medida detrás.

Localizarlo requirió construir un detector específico. El de salidas anómalas identifica truncamientos y expansiones por su longitud, y una frase traducida tiene longitud normal; ese punto ciego existía desde el principio y solo se hizo visible al evaluar un modelo con un modo de fallo distinto.

5.4. Comparativa de técnicas de adaptación

Técnica	Corpus	Palabras	WER base	WER téc.	Δ (pp)	IC 95 %	p	Signif.
Prompting contextual	tedx-es	532	10.15	9.96	-0.19	[-0.61, +0.00]	1.000	Sin potencia
Prompting contextual	teleconciencia-es	14.706	18.79	19.14	+0.35	[-0.52, +1.42]	0.012	En el límite
Prompting contextual	voxpath-es	771	9.99	9.60	-0.39	[-1.14, +0.45]	0.375	Sin potencia
Prompting contextual	voxpath-es-400	7.551	9.51	9.14	-0.37	[-0.76, -0.05]	0.230	En el límite
Post-corrección con LLM	teleconciencia-es	8.186	19.55	22.77	+3.23	[+2.66, +3.84]	0.000	Sí
Post-corrección con LLM	voxpath-es-400	12.951	9.63	10.81	+1.18	[+0.84, +1.54]	0.000	Sí
Post-corrección con LLM	voxpath-es	288	12.50	12.15	-0.35	[-1.69, +1.12]	1.000	Sin potencia
Fine-tuning con LoRA	voxpath-es-400	12.951	9.63	8.40	-1.23	[-1.75, -0.77]	0.000	Sí

Tabla 5.5: Efecto de cada técnica de adaptación sobre el WER (%), con diseño pareado. Δ negativo indica mejora. El intervalo de confianza se obtiene por bootstrap remuestreando clips; p corresponde al test de signos sobre los clips que cambian. La columna de significación distingue tres situaciones que no deben confundirse: *Sí* cuando ambos criterios coinciden, *En el límite* cuando discrepan entre sí, y *Sin potencia* cuando la muestra no alcanza las 5.000 palabras de referencia que exige la sección 5.1.2. Estas últimas no dicen que la técnica no funcione: dicen que con ese material no se puede saber, y por eso no se interpretan.

5.4.1. Prompting contextual

La técnica aporta al decodificador un glosario del dominio como contexto previo. El glosario se extrae de fragmentos disjuntos de los que se evalúan; construirlo con las transcripciones de evaluación sería una fuga de información y cualquier mejora medida sería falsa.

El resultado no es concluyente, y lo interesante es que no lo es de dos maneras incompatibles: sobre el corpus mexicano el efecto es de +0,35 puntos y sobre el peninsular de -0,37. En ambos casos los dos criterios estadísticos discrepan entre sí. Un efecto cuyo signo se invierte entre corpus y cuyas pruebas no coinciden no es un efecto pequeño: es la firma de la ausencia de efecto.

La métrica de terminología matiza esa conclusión. Sobre el corpus grande, el prompting aumenta la cobertura de términos del glosario en 1,42 puntos mientras reduce su precisión en 1,04: recupera términos que antes se perdían, y a cambio empieza a introducirlos donde no los hay. El desglose de errores lo confirma desde el otro lado, con 66 sustituciones menos pero 43 inserciones y 74 omisiones más. El modelo acierta más veces *qué* palabra es, y falla más veces *si* la palabra estaba.

5.4.2. Post-corrección con un modelo de lenguaje local

Un modelo de lenguaje corrige la transcripción ya producida. La salida del reconocedor queda fija, de modo que la única variable es la corrección.

La técnica degrada de forma significativa, y el resultado se replicó sobre dos corpus con características opuestas: +3,23 puntos sobre referencias con erratas y +1,18 sobre referencias verificadas manualmente. La diferencia de magnitud entre ambos es en sí misma informativa: buena parte del efecto medido inicialmente era un artefacto de la calidad de la referencia, y la cifra defendible es la del corpus limpio.

El mecanismo se observa directamente en las transcripciones. El modelo sustituye palabras por sinónimos más precisos, expande siglas, corrige la gramática del hablante y completa lo que interpreta como intención:

`cirugía → quirófano`

`Instituto de Tecnología de la UNAM → Universidad Nacional Autónoma
de México (UNAM)`

`atiendan a como un tipo terapia → atiendan cómo un tipo de terapia`

Cada una de esas ediciones mejora el texto como pieza escrita y lo empeora como transcripción; son objetivos en conflicto, y un modelo entrenado para generar lenguaje natural optimiza el primero.

El caso más ilustrativo es otro: en un fragmento, el corrector sustituyó el nombre de una persona por el nombre de una enfermedad, quedándose con el tema del programa. El

resultado es fluido, gramatical y completamente falso.

Conviene declarar una limitación de este resultado. La primera versión de las instrucciones dadas al corrector era puramente prohibitiva, y con ella la técnica quedó inerte: ningún fragmento modificado a nivel de palabra, y errores evidentes sin corregir. Añadir ejemplos de qué sí debía corregirse la activó. El experimento mide, por tanto, una implementación concreta y no la técnica en abstracto.

5.4.3. Ajuste fino con LoRA

Es la única de las tres que modifica los pesos del modelo, y la única que mejora de forma concluyente: 1,23 puntos de reducción del error, con intervalo de confianza $[-1,75, -0,77]$ y ambos criterios de acuerdo. El entrenamiento empleó 1.500 fragmentos (4,64 horas) durante 15,8 minutos, y produjo un adaptador de 18,9 MB frente a los 3 GB del modelo completo.

La partición de entrenamiento y la de evaluación provienen de divisiones oficiales distintas del mismo corpus, de modo que la separación no depende de un reparto propio.

El resultado exige un matiz. La caída del error sobre contenido crítico, de 17,52 % a 6,12 %, procede casi toda de los numerales, y el motivo se ve contando dígitos: la referencia escribe los números en letra y el modelo base los escribía en cifra. El ajuste fino aprendió esa convención de transcripción. Es adaptación real, pero de estilo y no de reconocimiento acústico. Un fragmento lo ilustra: donde la referencia dice **uno seiscientos ochenta y seis**, el modelo base escribió **1 686** y el ajustado **cero cero seis**; acertó el formato y falló el número, y ambas métricas lo premiaron. Descontando los numerales, la mejora sobre contenido crítico es real pero modesta: de 7,7 % a 6,2 %.

5.5. El WER no basta para evaluar accesibilidad

Cinco observaciones independientes, obtenidas a lo largo del trabajo y sobre modelos, corpus y técnicas distintos, apuntan a la misma conclusión: la métrica habitual del campo oculta precisamente los fallos que perjudican al usuario final.

Contenido crítico. Sobre el corpus grande completo, mil doscientos fragmentos, con una tasa de error de palabra del 18,61 %, que suena aceptable, el sistema pierde o altera el 28 % de las negaciones (134 de 477) e introduce trece que nadie pronunció. Perder una preposición se reconstruye por contexto; perder un *no* invierte la frase. El WER cuenta

ambos errores igual.

Alucinaciones fluidas. Los modelos producen ocasionalmente texto verosímil y falso en lugar de fallar de forma visible. Al tratarse de salidas gramaticales y coherentes con el dominio, ni el WER agregado ni el usuario final las detectan.

Terminología. El prompting contextual no mueve el WER pero sí recupera términos del glosario. Sin una métrica específica, la técnica habría parecido completamente inocua cuando en realidad tiene un efecto dirigido.

Duplicación por solapamiento. Un defecto de implementación que multiplicaba texto en las fronteras entre ventanas elevaba el WER por encima del 100 %, pero pasó inadvertido en las pruebas automáticas y solo se detectó al leer una transcripción real.

Dos arquitecturas que el WER declara equivalentes. Sobre habla espontánea, el modelo empleado y el transductor contrastado en la sección 5.3 no presentan diferencia detectable de tasa de error; sin embargo uno pierde el 35 % de las negaciones y el otro el 12 %. Es la observación más contundente de las cinco, porque descarta que el problema sea una peculiaridad de un modelo concreto: es una propiedad de la métrica.

Estas dos últimas cifras proceden de la submuestra de cuatrocientos fragmentos sobre la que se ejecutó el contraste de arquitecturas, y no de los mil doscientos del párrafo anterior; de ahí que la proporción de negaciones perdidas por el mismo modelo y el mismo corpus aparezca como 28 % en un caso y 35 % en el otro, y que su tasa de error figure como 18,61 % en la tabla 5.2 y como 19,55 % en la tabla 5.4. Son medidas distintas de la misma configuración sobre cantidades distintas de material, no una discrepancia. Se explicita porque el trabajo sostiene que el tamaño de la muestra forma parte de la identidad de un resultado, y eso obliga a declararlo también en la prosa.

De estas observaciones surgieron dos instrumentos complementarios: una métrica de cobertura y precisión sobre la terminología del dominio, y otra que contabiliza los errores sobre negaciones, numerales y cuantificadores, es decir, sobre las unidades cuya alteración cambia el significado y no solo la forma. A ellos se añadió después un tercero, el detector de fuga de idioma, por una razón que conviene retener: los dos primeros nacieron de fallos observados, y el tercero de descubrir que los dos primeros tenían un punto ciego. Ningún conjunto de métricas complementarias puede darse por cerrado.

5.6. Desarrollo de la aplicación

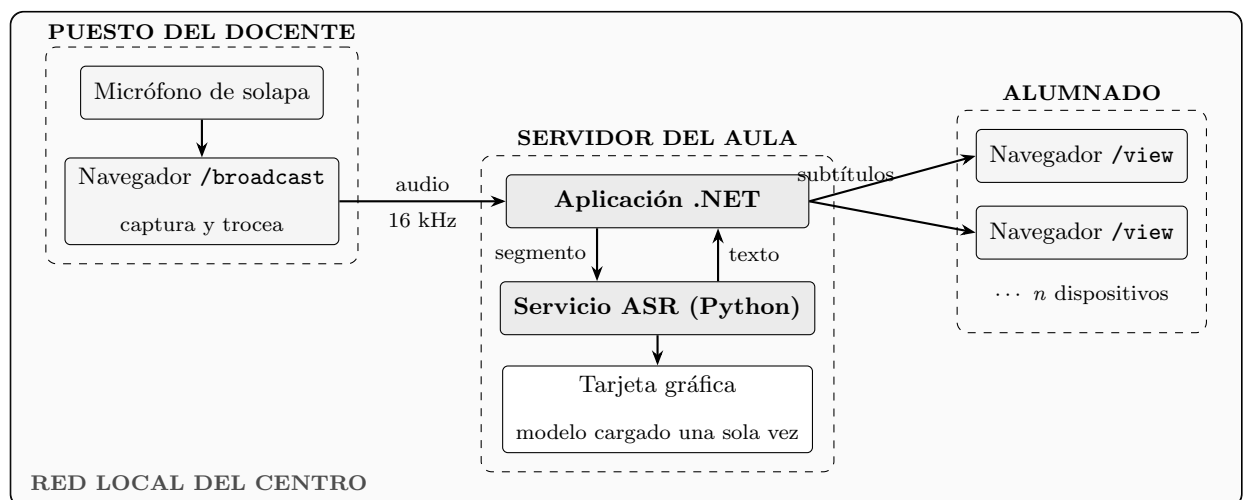
5.6.1. Arquitectura

El sistema separa estrictamente el núcleo investigador del aplicado: la aplicación depende de una interfaz de reconocimiento y no del código de los experimentos. Esa frontera permitió incorporar el adaptador ganador al sistema desplegado sin modificar una sola línea de la aplicación.

La topología es de un emisor y múltiples receptores. El docente captura desde su equipo; el alumnado únicamente visualiza. Todo el reconocimiento ocurre una vez en el servidor, con independencia del número de clientes conectados: un diseño donde cada cliente transcribiera exigiría una unidad de cálculo por alumno, lo que contradice el objetivo de que un centro pueda desplegarlo.

El acceso queda restringido a la red local por la propia aplicación, que rechaza cualquier conexión ajena a un rango privado. Se difunde audio de aula con voces identificables; la restricción no puede depender de que nadie configure mal un reenvío de puertos.

La figura 5.2 resume esa disposición.



Toda conexión ajena a un rango privado se rechaza. El audio no sale del centro y no se almacena: la transcripción vive en memoria mientras dura la clase.

Figura 5.2: Topología del sistema desplegado. El reconocimiento ocurre **una sola vez en el servidor** con independencia del número de dispositivos conectados; el alumnado solo visualiza, sin instalar nada ni conceder permisos.

5.6.2. La pila tecnológica y el puente entre los dos núcleos

La aplicación está escrita en C# sobre ASP.NET Core con Blazor Server, y el modelo se ejecuta en Python. Esa frontera no es una preferencia estética: la biblioteca de cálculo con soporte para la tarjeta gráfica empleada solo existe en Python, de modo que el reconocimiento no puede vivir dentro del proceso de la aplicación.

La unión es un servicio HTTP mínimo que carga el modelo una vez y expone una única operación: recibir audio y devolver texto. La restricción que lo gobierna es metodológica y no técnica: **usa exactamente la misma configuración de decodificación que los experimentos**. Si la aplicación decodificara de otro modo, las cifras medidas dejarían de describir el sistema desplegado, que es justamente el factor de confusión documentado en la sección 5.7.

Blazor Server se eligió porque resuelve gratis dos requisitos que de otro modo habrían exigido infraestructura propia. El primero es RF3: el alumnado no instala nada, porque toda la lógica de interfaz vive en el servidor y el navegador solo recibe actualizaciones del árbol de la página. El segundo es la difusión: el canal persistente que Blazor mantiene con cada cliente sirve para empujar los subtítulos, sin necesidad de montar un servicio de mensajería aparte. El precio es la contrapartida conocida de ese modelo: cada cliente consume estado en el servidor y la interfaz depende de la latencia de red. Resulta asumible con el número de clientes de un aula y con la red local como escenario.

5.6.3. Los componentes del núcleo

La lógica que decide qué se transcribe y cómo se muestra vive en una biblioteca sin dependencias de la interfaz ni del motor, lo que permite probarla sin arrancar ni el modelo ni el navegador. Seis piezas la componen.

La sesión de aula mantiene la clase en curso: el histórico de segmentos y la lista de clientes conectados. Es única para toda la instancia, porque representa la clase y no al usuario. De ahí sale RF5 casi sin código: un alumno que se incorpora tarde recibe el histórico retenido en el momento de conectarse.

El segmentador en vivo decide dónde cortar el audio, y es donde aterriza el hallazgo de la sección anterior. Sus valores por defecto proceden del barrido y no de la intuición: son los de la fila ganadora de la tabla 5.6, es decir tope de ocho segundos, mínimo de segundo y medio, pausa de 300 ms y umbral al 35 % del nivel de voz. El tope conviene

leerlo con cuidado, porque no significa que los segmentos duren ocho segundos: cortando en pausas casi nunca se alcanza, y de ahí que la duración media medida sea de 2,93 s. Es el valor que produce las cifras de latencia que reporta la sección 5.6.9, y una prueba automática comprueba que sigue siendo el que la aplicación carga, para que esta descripción no se separe del código. Hay una diferencia respecto al experimento que resultó tener consecuencias: en la grabación completa el umbral de silencio puede calcularse sobre el percentil 25 de las energías, porque la grabación entera contiene pausas; en vivo eso falla, ya que si el hablante lleva unos segundos sin parar, el percentil 25 ya es voz y el sistema se cree en silencio permanente. En vivo el umbral se define contra la *mediana* de las energías recientes. El fallo lo detectaron las pruebas automáticas, no el uso.

El fusionador de solapamiento recorta el mayor solape entre el final del texto ya emitido y el principio del nuevo. Existe por el defecto descrito más arriba, el que multiplicaba texto en las fronteras y llevaba el error por encima del 100 %. Gracias a él la aplicación puede conservar el solapamiento entre ventanas, que evita partir palabras en la frontera, sin pagar su coste.

El glosario resuelve RF4 marcando en el subtítulo los términos que el docente declara importantes. Es deliberadamente la versión léxica: coincidencia contra una lista aportada por el profesor, ignorando mayúsculas, tildes y puntuación, porque el motor no acentúa ni puntúa igual que un glosario escrito a mano. La detección semántica de conceptos es un problema abierto y quedó declarada fuera de alcance.

El filtro de red privada implementa RNF3 y RL3 rechazando toda conexión que no provenga de un rango privado. Si algún día hiciera falta acceso remoto, la vía correcta es una red privada virtual hasta el centro: el dispositivo entra en la red local y la comprobación lo acepta sin cambiar nada.

El control de acceso al puesto docente separa la vista de emisión de la de recepción mediante una clave compartida. No es autenticación de usuarios y no pretende serlo: la vista del alumnado es deliberadamente abierta, porque nadie debe pelearse con una contraseña para leer subtítulos, pero la de emisión activa el micrófono y difunde a toda la clase.

Esa asimetría tiene fundamento y no es solo comodidad. Según la evidencia recogida en la sección 2.1.2, la población que se beneficia de un subtítulo de aula es bastante mayor que el colectivo que lo justifica normativamente: incluye a quien está aprendiendo la lengua de la clase y, en menor medida, al alumnado sin ninguna necesidad específica. Un recurso que

hubiera que solicitar excluiría por diseño a casi todos ellos, y además obligaría a declararse a quien no quiera hacerlo. Abrir la vista de recepción es la decisión que convierte el sistema en una ayuda general en lugar de en un ajuste individual señalado.

La figura 5.3 recorre esas piezas en el orden en que actúan sobre un segmento de audio, e indica de dónde sale cada una. Merece leerse por la columna derecha más que por la izquierda: lo que distingue a este sistema no es la cadena de componentes, que es la esperable, sino que cada eslabón responde a una medición o a un fallo observado.

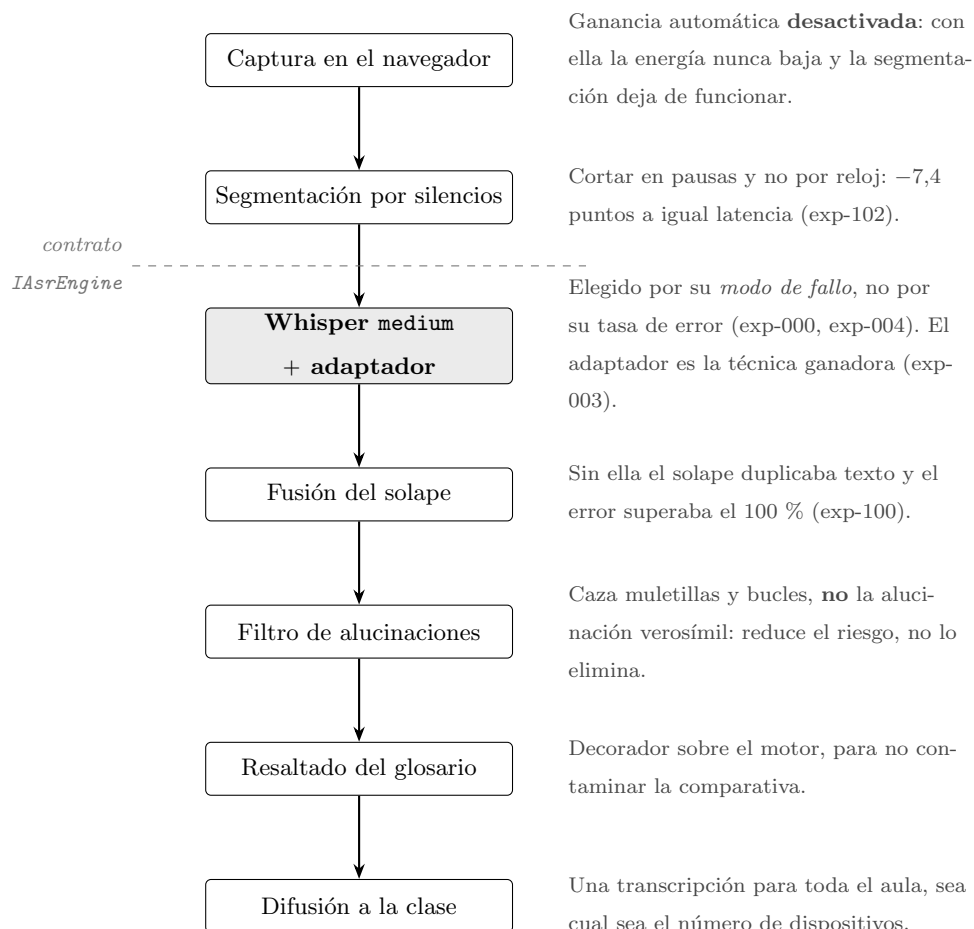


Figura 5.3: Recorrido de un segmento de audio y origen de cada decisión de diseño. La columna derecha indica qué experimento o qué incidente fijó cada etapa; ninguna se eligió por convención.

Una advertencia de diseño que costó tiempo: los motores se componen por decoración, de modo que el resaltado del glosario envuelve al motor real; eso hace que una comprobación de tipo concreto desde la interfaz **nunca se cumpla**. El diagnóstico de segmentación estuvo devolviendo ceros por ese motivo. Las capacidades opcionales viajan desde entonces

en interfaces propias que los decoradores reenvían explícitamente.

5.6.4. Troceado del audio: latencia frente a calidad

El subtitulado en vivo no puede esperar a que termine la intervención: hay que trocear. La pregunta es cuánto cuesta cada segundo de latencia que se ahorra.

Un barrido de tamaños de ventana mostró que trocear cada N segundos degrada de forma sustancial: con ventanas de tres segundos el error casi se duplica respecto a transcribir el fragmento completo. El motivo son los cortes arbitrarios, que caen en mitad de palabra o de sintagma justo donde el modelo necesita contexto.

Ese barrido reveló antes otra cosa. En su primera ejecución, las ventanas cortas producían tasas de error *por encima del 100 %*, lo que significa que el sistema emitía más errores que palabras tiene la referencia. La causa no era el modelo sino la implementación: cada ventana repetía el final de la anterior y ese texto se concatenaba dos veces, de modo que con ventana de un segundo y solape de medio, la mitad del contenido estaba duplicado. Aislado el defecto, el coste real de trocear con ventanas de un segundo baja del 105,32 % al 35,81 %. El panel izquierdo de la figura 5.4 conserva ambas curvas porque la distancia entre ellas es el resultado: casi todo lo que parecía coste de trocear era un error de fusión de texto.

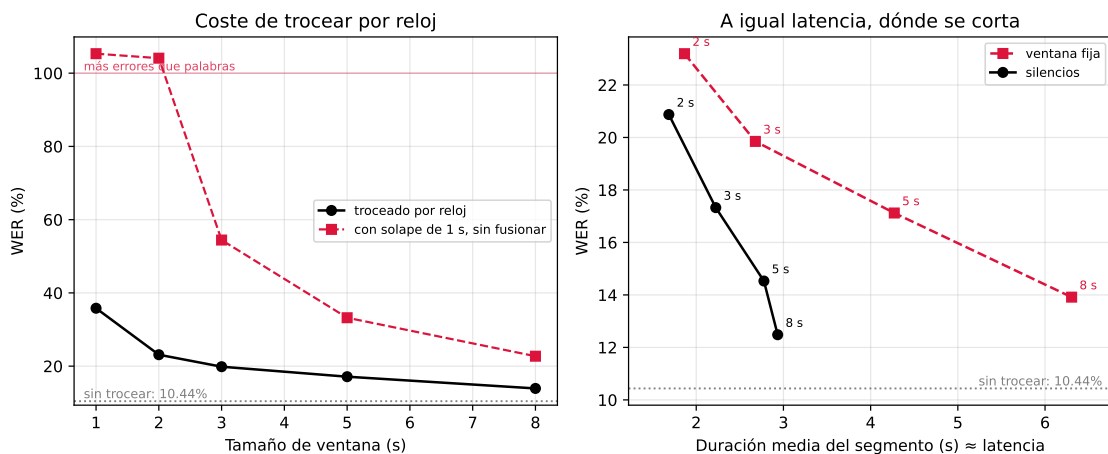


Figura 5.4: Izquierda: coste de trocear por reloj, con y sin la duplicación en las fronteras. Derecha: WER frente a latencia media según dónde se corte; las etiquetas indican el tope de duración. Generada desde los datos de ejecución.

La hipótesis de que el daño lo causan los cortes arbitrarios se contrastó cortando en los silencios en lugar de por reloj, con el mismo tope de duración en cada par para que la comparación no midiera simplemente la longitud del segmento. Los resultados aparecen en la tabla 5.6 y en el panel derecho de la figura 5.4.

Estrategia	Tope (s)	WER (%)	Dur. media (s)	Dur. p95 (s)	Segmentos
Ventana fija	2	23.19	1.87	2.00	277
Silencios	2	20.87	1.69	2.00	306
Ventana fija	3	19.85	2.68	3.00	193
Silencios	3	17.33	2.22	3.00	232
Ventana fija	5	17.12	4.27	5.00	121
Silencios	5	14.53	2.78	5.00	186
Ventana fija	8	13.92	6.30	8.00	82
Silencios	8	12.48	2.93	5.92	176
Sin trocear (techo)	—	10.44	—	—	40

Tabla 5.6: Segmentación por silencios frente a troceado por reloj, con el mismo tope de duración en cada par para que la comparación no mida simplemente la longitud del segmento. A latencia media equivalente (2.68 frente a 2.93 s), la segmentación por silencios reduce el error 7.37 puntos, a costa de que el percentil 95 pase de 3.00 a 5.92 s. Ese contraste se somete al mismo doble criterio que las técnicas de adaptación, sobre los mismos fragmentos: intervalo de confianza al 95 % de $[-9.76, -5.04]$ puntos por remuestreo de clips, y 30 fragmentos que mejoran frente a 2 que empeoran en el test de signos ($p = 2,5e - 07$).

La lectura por filas subestima el resultado, porque cada estrategia alcanza una duración media distinta con el mismo tope. La comparación que importa es a latencia media equivalente: ventana fija con tope de tres segundos produce segmentos de 2,68 s de media; segmentación por silencios con tope de ocho, de 2,93 s. Con esa diferencia de un cuarto de segundo, segmentar por silencios reduce el error 7,4 puntos y se queda a dos del techo que marca transcribir sin trocear.

Ese par se somete al mismo doble criterio que las técnicas de adaptación, y sobre los mismos fragmentos, porque de otro modo la comparación entre ambas magnitudes que sostiene la sección siguiente no sería legítima: no se puede contrastar un efecto medido con intervalo de confianza contra otro que solo tiene una diferencia de medias. El remuestreo por clips sitúa la diferencia en $[-9,76, -5,04]$ puntos, y el test de signos cuenta 30

fragmentos que mejoran frente a 2 que empeoran ($p \approx 2 \cdot 10^{-7}$). Ambos criterios coinciden. Conviene subrayar por qué el resultado es concluyente pese a las 1.466 palabras de referencia, que están muy por debajo del umbral de la sección 5.1.2: ese umbral se calculó para detectar diferencias del orden de medio punto, y esta es catorce veces mayor. La muestra basta para el tamaño del efecto que hay que resolver aquí, y no bastaría para el de las técnicas de adaptación. Lo que la muestra sigue sin sostener es la cifra exacta, que es propia de este material, según se declara en la sección 6.3.

El mecanismo se ve en las duraciones: con tope de ocho segundos, la segmentación por pausas produce segmentos de 2,9 segundos de media porque el tope casi nunca se alcanza. El resultado son segmentos cortos y completos, en lugar de cortos y partidos. En el panel derecho de la figura 5.4 esto aparece como dominancia estricta: la curva de la segmentación por pausas está por debajo de la otra en todo el rango de latencias, de modo que no hay ningún punto de operación en el que trocear por reloj sea preferible.

El precio es la previsibilidad: el percentil 95 pasa de 3,00 a 5,92 segundos. La latencia media mejora, pero el peor caso empeora; en accesibilidad esa distinción importa y ambas cifras deben reportarse.

Se comprobó además si arrastrar la transcripción anterior como contexto entre ventanas mejoraba los cortes. No se detecta ningún efecto: +0,14 puntos con ventana fija y +0,61 con segmentación por silencios, ambos con el signo equivocado para la hipótesis y ambos, sobre cuarenta fragmentos, muy por debajo de lo que ese material puede resolver (sección 5.1.2). La medición no permite por tanto afirmar que la técnica no sirva; permite afirmar que, si aporta algo, es menos de lo que aquí se puede medir.

La decisión de no arrastrar contexto no se apoya en esa cifra, que no daría para sostenerla, sino en dos razones que sí son independientes de ella. La primera es cualitativa y se observa en las transcripciones: el contexto propaga los errores del segmento anterior, de modo que una alucinación se hereda en lugar de extinguirse, y eso en accesibilidad pesa más que una décima de tasa de error. La segunda es que encaja con el mecanismo ya establecido: cuando los cortes caen en pausas, cada segmento es de por sí una unidad coherente, así que no hay información que el contexto pueda añadir. Ante un efecto no medible, se elige la opción que no puede propagar un fallo.

5.6.5. Requisito de hardware

Ejecutar el reconocimiento en unidad de procesamiento gráfico no es una comodidad sino un requisito. El modelo elegido alcanza 8,1 veces la velocidad del audio en tarjeta gráfica y solo 1,6 en procesador convencional; el modelo inmediatamente menor, 18,0 frente a 4,8. En ambos casos la aceleración ronda el factor cinco, de modo que no es un efecto del tamaño del modelo sino de dónde se ejecuta.

La cifra de 1,6 veces parece suficiente hasta que se recuerda que el sistema no transcribe una grabación sino un flujo. Cada segmento debe procesarse antes de que llegue el siguiente, y sobre ese margen tan estrecho cualquier variación, sea un segmento largo o cualquier otra carga del equipo, acumula retraso que ya no se recupera, porque el audio sigue llegando. Ejecutar en procesador convencional no es más lento: es inestable.

Esto acota RNF5 en términos concretos: el sistema requiere un equipo con tarjeta gráfica dedicada, que es un gasto asumible para un centro pero no es cualquier equipo de despacho. Y refuerza la decisión de arquitectura, porque ese equipo hace falta *una vez* para toda el aula: si cada dispositivo transcribiera, el requisito se multiplicaría por el número de alumnos y el sistema dejaría de ser desplegable.

5.6.6. Robustez frente al ruido de aula

Se caracterizó la degradación mediante ruido añadido de dos tipos: blanco, estacionario, y murmullo sintetizado superponiendo voces del propio corpus. Los resultados aparecen en la tabla 5.7.

Relación señal-ruido	Ruido blanco	Murmullo
Audio limpio	10.44	10.44
20 dB	10.50	11.66
15 dB	10.57	11.19
10 dB	13.23	12.41
5 dB	13.98	14.26
0 dB	16.85	36.02

Tabla 5.7: WER (%) según relación señal-ruido y tipo de ruido añadido. La mayor degradación con murmullo son 21.76 puntos entre 5 y 0 dB, frente a 2.86 puntos como peor salto con ruido blanco: el fallo no es gradual y depende del espectro del ruido, no solo de su energía.

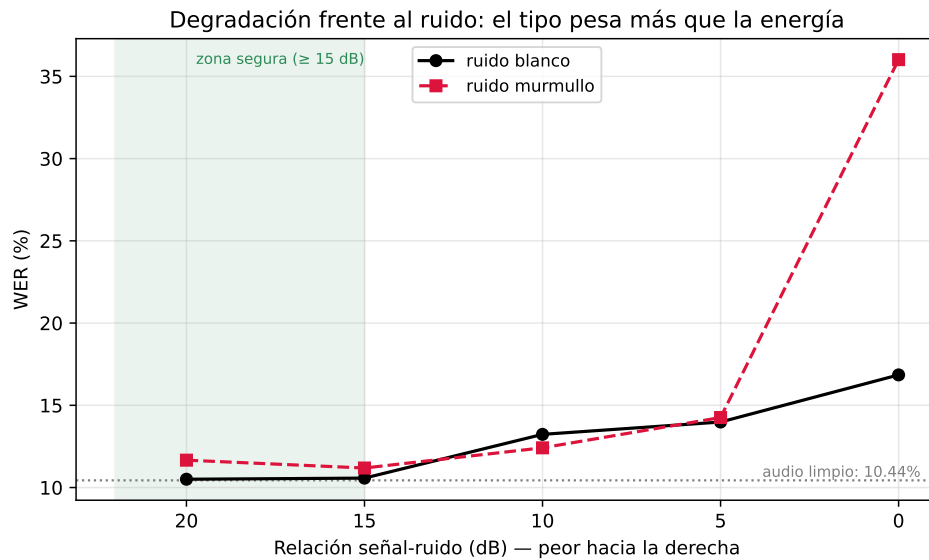


Figura 5.5: Degradación frente al ruido. La banda marca la zona en la que ningún tipo de ruido degrada más de 1,5 puntos, que es el ancho aproximado del intervalo de confianza que corresponde a una muestra de este tamaño según la sección 5.1.2; no se estima de nuevo sobre este barrido. Generada desde los datos de ejecución.

Hasta 15 dB la degradación se mantiene por debajo de lo que el propio barrido puede distinguir del ruido de medición, lo que sitúa un micrófono de solapa corriente en zona segura y permite formular el requisito de captación en términos concretos: RNF6 deja de ser “un micrófono adecuado” y pasa a ser “una captación que garantice 15 dB de relación señal-ruido en las condiciones de uso”, que es comprobable con un medidor.

Pero el fallo no es gradual, y esa es la lectura que la tabla esconde y la figura 5.5 muestra: entre 5 y 0 dB con murmullo el error salta 21,76 puntos de una vez, hasta multiplicar por tres y medio el del audio limpio, mientras que el ruido blanco al mismo nivel apenas lo altera, con 2,86 puntos como peor salto. La diferencia confirma el mecanismo: lo que arruina el reconocimiento no es la energía del ruido sino que comparta espectro con la voz. Seis voces de fondo enmascaran lo que un ventilador no, y por eso los supresores de ruido convencionales, diseñados para ruido estacionario, no resuelven el problema del aula.

La consecuencia práctica para el despliegue es que el sistema no avisa de que se está degradando. Pasa de funcionar bien a fallar en un margen de cinco decibelios, de modo que una clase que se anima, con el ruido de fondo subiendo mientras la voz del docente no, puede cruzar el umbral sin que nada lo señale. Es un argumento a favor del micrófono

de proximidad y en contra del micrófono ambiental, más allá de la diferencia media de calidad.

5.6.7. Contención de alucinaciones en el sistema desplegado

La elección de modelo se hizo con el criterio de minimizar las alucinaciones, pero minimizarlas no es eliminarlas, y el uso real lo demostró de inmediato. Con el micrófono abierto y el docente en pausa, el sistema emitió «¡*Suscríbete al canal!*» y, en otra ocasión, un bucle de la conjunción *y* repetida. El motivo es conocido: el modelo se entrenó con enormes cantidades de subtítulos de vídeo, de modo que ante silencio o audio ininteligible tiende a emitir las frases más frecuentes de ese material en lugar de callar.

Es el fallo más grave que este sistema puede cometer. La salida es gramatical, se lee con naturalidad y el alumno con discapacidad auditiva no tiene ninguna forma de detectar que no corresponde a lo dicho: es exactamente la asimetría entre subtítulo ausente y subtítulo inventado que atraviesa todo el trabajo. Por eso la aplicación incorpora un filtro que descarta el segmento antes de difundirlo, sobre los dos patrones que sí son reconocibles por su forma: las muletillas propias del corpus de subtítulos, y los bucles de repetición de una misma palabra. El umbral de repetición se fijó en cinco y no en tres porque «*no, no, no*» es habla legítima y frecuente.

Conviene declarar con precisión qué *no* hace, porque presentarlo como una garantía sería peor que no tenerlo. No detecta alucinaciones plausibles en el dominio, y esa limitación está confirmada en uso real: ya con el filtro activo, el sistema transcribió «*me he mantenido en silencio durante la noche*» donde el docente había dicho «*durante 30 o 40 segundos*». Esa salida es gramatical, encaja en el contexto y tiene longitud normal; ninguna heurística de superficie la distingue de habla real. Detectarla exigiría contrastar la hipótesis con el audio, que es un problema distinto y abierto.

El filtro es, por tanto, una reducción de riesgo y no una garantía, y así debe presentarse a quien despliegue el sistema. Su valor real es doble: elimina la clase de alucinación que aparece en los silencios, que es la más frecuente en un aula por la sencilla razón de que un docente pasa buena parte de la clase sin hablar; y deja constancia explícita de que el residuo existe.

5.6.8. La interfaz

Las dos vistas son deliberadamente asimétricas. La de recepción muestra el texto y ofrece un único control, el tamaño de la letra (RF6). No es una comodidad: el sistema se dirige a personas con una discapacidad sensorial, y no cabe suponer que quien lo desarrolla pueda fijar un tamaño válido para todas ellas ni para todas las distancias de lectura de un aula.

La de emisión concentra el resto: captura del micrófono, edición del glosario de la sesión, latencia medida y un diagnóstico de segmentación que muestra cuántos cortes se han cerrado *por pausa* frente a cuántos *por agotar el tope*. Este último contador merece explicación, porque no es información de depuración sino la manifestación visible del incidente descrito en la sección 5.7: cuando el preprocesado del navegador sabotea la detección de silencios, la proporción se desploma hacia el tope. Exponerla en la interfaz convierte un fallo silencioso en un fallo observable por quien usa el sistema, sin necesidad de leer un registro.

5.6.9. Latencia del circuito completo

RNF1 exige reportar la latencia con su valor típico y su peor caso, y RF7 que el sistema la informe. La latencia se descompone en tres sumandos, y conviene separarlos porque solo uno es reducible por ingeniería:

1. **La espera a que el segmento se cierre**, que domina el total: 2,93 s de media y 5,92 s en el percentil 95 con la configuración adoptada. No depende del modelo ni del equipo, sino de cuándo hace una pausa quien habla.
2. **La inferencia**, entre 0,5 y 1,0 s por segmento según su duración, con el modelo elegido sobre la tarjeta gráfica empleada.
3. **El circuito de red y renderizado**, que la aplicación mide directamente en el navegador marcando el instante en que salió el audio y el instante en que vuelve el subtítulo, con un único reloj para que la medida no dependa de la sincronía entre equipos.

De ahí una conclusión de diseño que conviene explicitar: **un modelo más rápido no rescata un sistema en vivo mal segmentado**. Aunque la inferencia fuera instantánea,

seguiría habiendo que esperar a que el docente termine la frase, y ese término domina el presupuesto. Es la misma conclusión de la comparativa, vista desde el otro lado.

Queda una carencia que se declara como tal: la instrumentación existe y la aplicación muestra la cifra en cada sesión, pero no se ha registrado una campaña sistemática de medidas del circuito completo en condiciones de aula, de modo que el tercer sumando se conoce por observación durante el desarrollo y no por una distribución medida. Es la única magnitud del trabajo que se reporta sin respaldo experimental propio, y así consta en la sección 6.3.

5.6.10. Verificación de la aplicación

La lógica del núcleo está cubierta por 88 pruebas automáticas que se ejecutan con un solo comando y no requieren ni modelo ni navegador. No son pruebas de trámite: cada grupo fija un invariante que se rompió al menos una vez durante el desarrollo, y su reparto es revelador de dónde estuvieron los problemas.

El grupo más numeroso, con 21 casos, es el del filtro de red privada, que recorre los rangos privados, el bucle local y el enlace local frente a direcciones públicas. Es desproporcionado respecto al tamaño del código que verifica, y a propósito: implementa un requisito legal, de modo que un falso negativo no es un fallo funcional sino una brecha de protección de datos.

Le siguen las 15 del filtro de alucinaciones, que fijan tanto lo que debe descartarse como lo que **no**: cinco repeticiones de una palabra sí, tres no, porque «no, no, no» es habla legítima. Siguen las 14 del fusionador de solapamiento, que impiden la duplicación en las fronteras, el defecto que llegó a producir un error superior al 100 %. Las 12 del glosario incluyen la que cierra el incidente de la eñe: comprueban que *año* y *ano* no colisionan al normalizar, que es lo que ocurre si la protección se aplica después de descomponer en Unicode y no antes. Las nueve del segmentador cubren el criterio de corte, y entre ellas están la que detectó que el umbral de silencio debía calcularse contra la mediana y no contra el percentil 25, las que fijan el reparto entre cortes por pausa y por tope que la interfaz muestra al docente, y la que comprueba que un segmento sin voz suficiente se descarta antes de llegar al modelo. Las siete de la sesión cubren la incorporación tardía de un cliente, y las cinco del control de acceso al puesto docente incluyen el camino que deja la emisión abierta cuando no hay clave configurada.

Las cinco últimas son de otra clase: verifican el fichero de configuración contra el código.

Dos comprueban que ninguna clave de comentario se cuele donde el sistema de registro las interpretaría como niveles, un fallo que dejó la aplicación en bucle de reinicio dentro del contenedor sin que el error apuntara al fichero. Las otras tres comprueban que la sección de segmentación llega al objeto que la usa, que sus valores son los documentados en esta memoria, y que el glosario configurado no acaba vacío. Existen porque el enlace de configuración no falla cuando una clave no coincide: usa el valor por defecto y sigue, sin excepción ni aviso. Dos secciones enteras quedaron así inertes durante un tiempo, entre ellas la del glosario, que dejó RF4 sin marcar nada sin que nada lo delatara.

5.6.11. Despliegue y portabilidad

El sistema se despliega con dos contenedores: el que reconoce, que necesita tarjeta gráfica, y el que sirve la interfaz, que no. El fabricante de la tarjeta se elige mediante perfiles, porque las dos formas de exponer una GPU a un contenedor son incompatibles entre sí y no pueden coexistir en la misma definición.

Dos decisiones de esa configuración merecen constar. La primera es que ambos contenedores comparten la red del anfitrión en lugar de publicar puertos: los núcleos recientes de la distribución empleada no incluyen los módulos de compatibilidad de cortafuegos que la redirección de puertos necesita, y el aislamiento de red no aportaría nada aquí, porque el sistema sirve a la red local de todos modos y la restricción de alcance la impone la aplicación. El reparto resultante es el deseado: el reconocedor escucha solo en bucle local y la aplicación en todas las interfaces. La segunda es que las cachés de modelo y de núcleos compilados se persisten en volúmenes; sin ellos, cada arranque volvería a descargar varios gigabytes y a recompilar los núcleos de cálculo, porque la plataforma empleada no distribuye binarios precompilados para esa generación de tarjeta.

El servicio de reconocimiento selecciona acelerador en tiempo de arranque entre tarjeta dedicada, acelerador de Apple Silicon y procesador convencional, forzando precisión simple en el segundo caso porque su implementación no cubre la precisión reducida en todas las operaciones del modelo.

Esa portabilidad tiene un límite que conviene declarar donde se describe, y no solo en el manual de despliegue: sobre la arquitectura de Apple, común a sus portátiles y a sus equipos de sobremesa, las dos vías son incompatibles entre sí. Los contenedores en ese sistema corren dentro de una máquina virtual que no expone el acelerador, de modo que el despliegue contenerizado quedaría en procesador, con el factor de tiempo

real insuficiente que mide la sección 5.6.5. La vía válida allí es la ejecución nativa. Y el rendimiento de ese acelerador no está medido en este trabajo: se ha comprobado que el servicio lo detecta y transcribe, pero el barrido de dispositivos comparó únicamente tarjeta dedicada y procesador. Esa portabilidad es de la *aplicación*, no de los experimentos: los programas de investigación siguen fijando tarjeta dedicada o procesador, porque se ejecutan en el equipo de desarrollo y ampliar ahí la selección cambiaría resultados ya medidos.

5.6.12. Incorporación de la técnica ganadora

El octavo objetivo exigía que la técnica ganadora entrara en el sistema desplegado sin modificar la aplicación. Se cumple literalmente: el servicio de reconocimiento acepta la ruta de un adaptador y lo aplica sobre el modelo base al cargarlo, de modo que incorporar el resultado del ajuste fino es un argumento de arranque. La aplicación no se entera, porque depende de la interfaz de reconocimiento y no de lo que haya detrás. La configuración de despliegue expone esa ruta como variable, con el modelo base como valor por defecto.

Es la justificación práctica de la frontera entre los dos núcleos: permitió enseñar una demo funcionando mientras la comparativa seguía en curso, y sustituir el modelo por el adaptado al terminarla sin tocar una línea de la aplicación.

5.7. Incidentes metodológicos

Varios de los resultados anteriores estuvieron a punto de ser erróneos. Se documentan porque comparten una característica que los hace peligrosos: en todos los casos el sistema seguía funcionando y produciendo cifras verosímiles.

La configuración de decodificación como factor de confusión. Con decodificación voraz, uno de los modelos truncaba seis de cada cuarenta fragmentos y llegaba a emitir frases plausibles del dominio en lugar del contenido real; su tasa de error subía de 11,5 % a 24,2 %. Comparar técnicas sin fijar esta configuración habría medido artefactos de decodificación en lugar del efecto de la adaptación. Desde entonces, la decodificación forma parte del nombre de cada fichero de resultados.

Sobrescritura del corpus. Al ampliar un corpus de 40 a 1.200 fragmentos se sobrescribió su manifiesto; los resultados anteriores seguían apuntando a un fichero cuyo contenido ya era otro, y nada lo delataba. La corrección fue doble: registrar el resumen criptográfico del manifiesto en cada resultado, para detectar la discrepancia, y permitir

que varias muestras de la misma fuente convivan, para prevenirla.

El mismo fallo, un nivel más arriba. La corrección anterior resolvió el manifiesto pero no el *resultado*. La identidad de un resultado se compone de modelo, corpus y decodificación, y no incluye el tamaño de la muestra; al reevaluar el modelo elegido sobre la ampliación, su fichero sobrescribió al de la muestra reducida. Las dos cifras eran correctas por separado, pero el generador de tablas las ponía en la misma columna que las de los otros dos modelos, medidos sobre 40 fragmentos: una comparación de 1.200 contra 40 presentada como si fuera pareada. Se detectó al auditar la memoria contra sus datos, no al producirla. La corrección aplicada es que el generador arrastra el número de fragmentos por fila, comprueba que todas las filas coincidan y, si no, lo advierte en la consola y en el propio pie de la tabla. La solución de fondo es la que ya se aplicó a otro corpus: **que la muestra forme parte del nombre**, de modo que dos tamaños no puedan colisionar.

El patrón que comparten ambos incidentes merece enunciarse, porque es la lección general del trabajo: cada vez que se añade un eje de variación, sea el corpus, la decodificación, el solapamiento o el tamaño de la muestra, hay que añadirlo a la *identidad* del resultado. Omitirlo no produce un error: produce una comparación inválida con aspecto de tabla.

Un entrenamiento roto que no daba error. El primer ajuste con LoRA terminó correctamente, guardó su adaptador y produjo métricas evaluables, con la función de pérdida estancada en 6,87 y subiendo entre épocas. La causa era que el preparador de lotes descartaba un identificador de token equivocado, de modo que el decodificador recibía duplicado el marcador de inicio. Corregido, la pérdida cayó a 0,21. De no haberse detectado, la evaluación habría mostrado que el ajuste fino destroza el modelo, con intervalo estrecho y significación respaldando un artefacto.

Ajustar sobre referencias defectuosas. Uno de los corpus disponibles omite las tildes de forma sistemática. Entrenar sobre él habría enseñado al modelo a no acentuar y, medido contra esas mismas referencias, la tasa de error *habría mejorado*: se habría optimizado hacia el error con la métrica dando la razón.

La ñe desaparecida. Para comparar cadenas ignorando la acentuación, el procedimiento habitual es descomponer el texto en Unicode y descartar los diacríticos. Hecho así sobre español, la ñe se convierte en ene, porque en forma descompuesta *ñ* ya es *n* más una tilde combinante. La comprobación que pretendía protegerla se escribía *después* de descomponer, de modo que no se cumplía nunca. En español eso no es cosmético: cambia

la palabra. El error se cometió dos veces, una en cada lenguaje del proyecto, y en ambos casos el programa funcionaba y devolvía cifras. La *eñe* se sustituye ahora por un centinela *antes* de descomponer.

Configuración que no llega y contadores que no cuentan. Al auditar la memoria contra el código aparecieron tres piezas que declaraban una intención sin ejecutarla, y ninguna emitía error. Dos secciones del fichero de configuración habían quedado inertes al traducir los identificadores del código al inglés sin traducir las claves del fichero: el enlace no falla cuando un nombre no coincide, se queda con el valor por defecto y sigue. La del glosario dejaba la lista vacía en todos los despliegues, de modo que el resaltado de terminología no marcaba nada; la de segmentación hacía que ajustar el umbral de silencio no tuviera ningún efecto, precisamente lo que un comentario del propio fichero recomendaba hacer si el diagnóstico se desplomaba hacia el tope. Y ese diagnóstico tampoco funcionaba: sus dos contadores estaban declarados, documentados y expuestos en la interfaz, pero nunca se incrementaban, así que mostraban cero indefinidamente. Junto a ellos, una comprobación que debía descartar los segmentos sin voz suficiente antes de enviarlos al modelo estaba escrita, comentada y sin usar.

Las tres comparten la forma de los demás incidentes de esta sección: código que se ejecuta sin fallar y produce un resultado plausible, en este caso un glosario vacío, un ajuste sin efecto y un contador a cero, ninguno de los cuales se distingue del comportamiento correcto sin ir a comprobarlo. Y las tres tienen la misma corrección, que es la general del trabajo aplicada al código en lugar de a los datos: una prueba automática que ata la descripción a lo que el sistema hace de verdad. Las cuatro pruebas de configuración descritas más arriba existen por esto.

Un preprocesado que anula la segmentación. Al probar la aplicación con micrófono real, casi todos los segmentos se cerraban por agotar el tope en lugar de por pausa. La causa no estaba en el algoritmo sino en el control automático de ganancia del navegador: al callar el hablante sube la ganancia, amplifica el ruido de fondo y la energía nunca descende lo suficiente. Es un efecto que ningún experimento sobre audio grabado podía detectar, porque solo existe cuando hay un micrófono y un navegador de por medio.

Este último incidente ilustra la relación entre los dos núcleos del trabajo mejor que cualquier argumento: la investigación proporcionó a la aplicación su criterio de diseño, y la aplicación descubrió una condición sin la cual ese criterio no se cumple.

6. Conclusiones y Trabajo Futuro

6.1. Conclusiones

6.1.1. Sobre las técnicas de adaptación

De las tres técnicas evaluadas, solo mejora la que modifica los pesos del modelo. El ajuste fino con LoRA reduce el error 1,23 puntos con ambos criterios estadísticos de acuerdo; el prompting contextual no produce efecto detectable, con signo distinto en cada corpus y pruebas que discrepan entre sí; y la post-corrección con un modelo de lenguaje local lo empeora de forma significativa y replicada.

La conclusión sobre la post-corrección merece detenerse, porque contradice la intuición del campo. Un modelo de lenguaje debería poder reparar errores de reconocimiento usando el contexto, y sin embargo degrada. El mecanismo observado explica por qué: el corrector optimiza la calidad del texto como pieza escrita, sustituyendo palabras por sinónimos más precisos y corrigiendo la gramática del hablante, mientras que una transcripción debe reflejar lo dicho. Son objetivos en conflicto, y el modelo optimiza el que le corresponde por entrenamiento.

Sobre LoRA cabe un matiz que conviene no omitir: buena parte de su mejora consiste en aprender la convención de escritura del corpus de ajuste, no en reconocer mejor el audio. Descontado ese efecto, la ganancia sigue existiendo pero es modesta.

6.1.2. Sobre dónde está realmente el margen de mejora

El resultado más relevante del trabajo no procede de la comparativa de técnicas sino de su comparación con las decisiones de ingeniería del sistema. Segmentar el audio por silencios en lugar de por reloj reduce el error 7,4 puntos a igual latencia media, con intervalo de confianza $[-9,76, -5,04]$ y el test de signos de acuerdo; seis veces más que la mejor de las técnicas de adaptación, y a coste de cómputo prácticamente nulo. La comparación entre ambas magnitudes es legítima porque las dos se han medido con el mismo instrumento y bajo el mismo criterio, no solo como diferencia de medias.

Esto invierte la premisa de la que partía el trabajo. Se asumía que el margen estaba en adaptar el modelo al dominio educativo; los datos indican que, con los modelos actuales, el margen está en cómo se le entrega el audio. Un sistema de subtitulado en vivo que trocee

mal desperdicia más calidad de la que cualquier técnica de adaptación puede recuperar.

6.1.3. Sobre la evaluación

La tasa de error de palabra es insuficiente para valorar un sistema de accesibilidad, y el trabajo aporta cinco evidencias independientes de ello. La más contundente: con un 18,61 % de error, cifra que suena aceptable, el sistema pierde o altera el 28 % de las negaciones e introduce trece que nadie pronunció. Un alumno con discapacidad auditiva lee entonces lo contrario de lo explicado, sin ninguna señal de que algo falla.

De ahí se derivan dos instrumentos que se proponen como complemento: una métrica de cobertura y precisión sobre la terminología del dominio, y otra que contabiliza los errores sobre negaciones, numerales y cuantificadores. Ambas detectan efectos que el WER agregado no muestra, y en un caso concreto invierten la valoración de una técnica.

6.1.4. Sobre la elección del modelo base

La comparativa de técnicas se construyó sobre un único modelo, elegido entre variantes de su misma familia; eso responde cuál de ellas, no por qué esa familia. Contrastarlo con una arquitectura distinta, un transductor multilingüe reciente, produjo tres resultados que conviene leer juntos (tabla 5.4).

El primero es que la ventaja en tasa de error existe pero no es general: 2,31 puntos sobre habla parlamentaria leída y ninguna diferencia detectable sobre habla espontánea. El segundo refuerza la conclusión anterior sobre la evaluación: con tasas de error indistinguibles entre sí, un modelo pierde el 35 % de las negaciones y el otro el 12 %, medido sobre la submuestra de cuatrocientos fragmentos del contraste y no sobre el corpus completo del que sale la cifra del apartado anterior. La insuficiencia del WER deja así de ser atribuible a una peculiaridad de un modelo concreto y se muestra como una propiedad de la métrica.

El tercero decide. El modelo alternativo no falla truncando sino traduciendo: ante habla espontánea española produjo frases completas en inglés, y alternancias de idioma dentro de una misma oración. Trece casos sobre ochocientos clips, frente a ninguno del modelo empleado. Es el mismo criterio que ya había descartado a la variante más rápida de la familia original, aplicado ahora contra el candidato que ganaba en las métricas: en accesibilidad el modo de fallo pesa más que unas décimas de error, porque una transcripción ausente se advierte y una traducida al inglés se lee como si el docente hubiera cambiado de idioma.

Pesa además que el fallo no admite corrección por configuración: esa arquitectura no expone ningún mecanismo para imponer el idioma de salida, mientras que al modelo empleado sí se le fija y no fugó ni una vez en los mismos clips. La capacidad de forzar el idioma pasa así de comodidad a requisito para este tipo de sistema.

Detectarlo exigió un instrumento nuevo, y ese es un resultado en sí mismo: el detector de salidas anómalas del que se disponía las identifica por su longitud, y una frase traducida tiene longitud normal. El punto ciego llevaba ahí desde el principio.

6.1.5. Sobre la validez de las mediciones

Ocho incidentes documentados en la sección 5.7 comparten una característica: en todos ellos el sistema seguía funcionando y produciendo cifras verosímiles. Ninguno se manifestó como un error.

La conclusión práctica es que un procedimiento experimental en este campo necesita comprobaciones que no forman parte del procedimiento habitual: verificar que la función de pérdida esté en el rango esperado antes de evaluar un modelo ajustado; auditar a mano una muestra de las referencias antes de adoptar un corpus; incorporar la configuración de decodificación a la identidad del resultado; y registrar el contenido exacto del corpus empleado, no solo su nombre.

Hay además una conclusión más general, y es la que este trabajo defendería ante la pregunta de qué aporta metodológicamente. Todos los incidentes son la misma clase de fallo: un eje de variación que existía en el experimento pero no en la *identidad* del resultado. La decodificación, el contenido del corpus, el solapamiento entre ventanas y el tamaño de la muestra entraron en esa identidad uno a uno, y siempre después de que su ausencia produjera una cifra falsa. La regla que se deriva es esta: cualquier eje que se introduzca debe incorporarse al nombre del resultado, no solo a su contenido. Es barata de aplicar y cara de descubrir, porque su incumplimiento nunca se manifiesta como un fallo sino como una tabla.

6.2. Verificación de los objetivos

Los criterios de cumplimiento se fijaron en el capítulo 4 antes de ejecutar los experimentos, precisamente para no poder ajustarlos después a los resultados. La tabla 6.1 los contrasta con lo obtenido; la trazabilidad de los requisitos de los que se derivan está en

la sección 3.3. Se marca como parcial lo que se cumplió con una salvedad, en lugar de redondear al alza.

Obj.	Estado	Evidencia
1	Parcial	Toda cifra se regenera con un comando y todo resultado registra su configuración. La procedencia completa (commit, estado del árbol y resumen criptográfico del corpus) cubre los once resultados posteriores al mecanismo, no los cuarenta y uno anteriores; para estos, la comprobación equivalente está automatizada. Desglose en el apéndice A.
2	Cumplido	Las tres técnicas se miden sobre el mismo corpus, con la misma decodificación y el doble criterio de significación (tabla 5.5).
3	Cumplido	El tamaño de muestra se justifica con un cálculo previo; tres de las ocho mediciones quedan marcadas sin potencia en la propia tabla y no se interpretan.
4	Cumplido	Tres métricas complementarias, con evidencia de que los fallos que detectan existen en el material analizado.
5	Cumplido	Un cliente de la red visualiza los subtítulos sin ejecutar cálculo ni conceder permisos; el reconocimiento ocurre una vez para toda el aula.
6	Cumplido	La segmentación se elige con barrido propio (exp-100 y exp-102), no por convención; el punto de operación se justifica con la curva completa.
7	Cumplido	La curva de degradación permite expresar RNF6 en decibelios en lugar de como recomendación genérica.
8	Cumplido	El adaptador entra en el sistema desplegado como argumento de arranque del servicio, sin modificar la aplicación (sección 5.6.12).
RNF1	Parcial	Los dos primeros sumandos de la latencia están medidos; el circuito de red y renderizado está instrumentado pero sin campaña sistemática de medida.

Tabla 6.1: Verificación de los objetivos frente a los criterios fijados de antemano.

Siete de los ocho objetivos específicos se consideran alcanzados sin salvedades. El primero queda como parcial, y conviene explicar por qué se marca así en lugar de darlo por bueno: el criterio exigía que *cualquier* cifra publicada registrase el commit y el contenido exacto del corpus, y eso es cierto del procedimiento tal como está hoy pero no de todo el archivo, porque el mecanismo se construyó a mitad del trabajo y como respuesta a un fallo. Redondearlo al alza habría sido especialmente incongruente en un trabajo cuya aportación metodológica consiste en desconfiar de las afirmaciones que un sistema hace

sobre sí mismo. La otra casilla parcial no corresponde a un objetivo sino a un requisito no funcional, y se declara aquí en lugar de omitirse porque afecta a una cifra que el trabajo reporta.

Conviene además señalar lo que el cumplimiento *no* significa. El segundo objetivo se alcanzó y su resultado es mayoritariamente negativo: dos de las tres técnicas no mejoran, y una empeora. Eso era una posibilidad prevista desde la planificación, y el criterio de cumplimiento se redactó para que un resultado negativo bien medido contara como objetivo alcanzado. Lo contrario habría creado un incentivo para forzar la interpretación de los datos.

6.3. Limitaciones

Ninguno de los corpus es una clase real. Es la limitación de fondo. Los materiales empleados se aproximan al registro de una clase magistral, pero ninguno lo es. Las conclusiones sobre adaptación al dominio educativo son, en rigor, conclusiones sobre adaptación a los dominios sustitutos disponibles. El corpus objetivo, un repositorio universitario con transcripciones manuales, no está publicado y requiere una solicitud institucional que queda redactada y dirigida, pero cuya tramitación excede el plazo de este trabajo.

La comparación entre arquitecturas no tiene la decodificación igualada. No es un descuido subsanable: un transductor emite un símbolo por trama y carece del bucle autorregresivo sobre el texto en el que el modelo empleado aplica su reintento por temperatura, de modo que no existe una configuración común sobre la que igualarlos. Cada sistema se evaluó en su configuración estándar y la comparación es, en rigor, entre sistemas completos y no entre arquitecturas con el resto controlado.

Los resultados miden implementaciones, no técnicas. Se hizo evidente con la post-corrección: una primera versión de las instrucciones dejaba la técnica inerte, y otra la activaba. Un prompt distinto, un modelo mayor o un ajuste de hiperparámetros podrían alterar las conclusiones.

El detector de actividad de voz es de energía. Un detector entrenado sería más robusto frente al ruido de aula, y la práctica del sector indica que la combinación de ambos reduce sustancialmente las falsas activaciones. La medición realizada justifica introducirlo, pero no se ha implementado.

El ruido se añadió de forma sintética. Falta la reverberación, que es un factor

independiente y posiblemente más dañino que el ruido aditivo en un aula real.

La aplicación no se ha evaluado con usuarios. Se ha verificado su funcionamiento técnico y se ha medido su latencia, pero no se ha comprobado si el resultado es útil para quien debe leerlo. Es la limitación más incómoda del trabajo, porque la literatura sobre subtitulado accesible (Kafle y Huenerfauth, 2017) indica precisamente que las métricas automáticas predicen mal esa utilidad, y este trabajo lo sostiene con datos propios sin llegar a cerrar el círculo. Vale tanto para el colectivo que motiva el trabajo como para el alumnado que está aprendiendo la lengua, cuya inclusión en la sección 2.1.2 se apoya en literatura ajena y no en medición propia.

La latencia del circuito completo no tiene campaña de medida. Los dos sumandos dominantes están caracterizados experimentalmente, y la aplicación instrumenta el tercero y lo muestra en cada sesión, pero no se ha recogido una distribución de medidas en condiciones de aula. La cifra que el trabajo defiende como latencia del sistema es, por tanto, la suma de dos términos medidos y uno observado.

El filtro de alucinaciones es heurístico por construcción. Cubre los dos patrones reconocibles por su forma y no puede cubrir el tercero, la alucinación plausible en el dominio, cuya existencia está confirmada en uso real (sección 5.6.7). Detectarla exigiría contrastar la hipótesis con el audio, que es un problema distinto.

Las mediciones del núcleo aplicado se apoyan en un solo corpus y un solo modelo. Los barridos de ventana, segmentación y ruido se ejecutaron sobre cuarenta fragmentos de habla parlamentaria peninsular. La magnitud de la ventaja de segmentar por pausas es lo bastante grande como para que su signo no dependa de esa elección, pero la cifra concreta de 7,4 puntos sí es propia de ese material.

El sistema depende de un único servidor. La difusión se resuelve en memoria, lo que ata el despliegue a un proceso. Escalar a varios exigiría una infraestructura adicional; para un aula no aplica, pero condiciona cualquier extensión a nivel de centro.

6.4. Trabajo futuro

Corpus educativo real. Es la continuación prioritaria: repetir la comparativa sobre clases grabadas convertiría las conclusiones actuales, obtenidas sobre dominios sustitutos, en conclusiones sobre el dominio objetivo.

Detección de actividad de voz entrenada. La segmentación resultó ser la palanca

más eficaz del sistema, y actualmente se apoya en el método más simple posible. Mejorarla es la vía con mayor rendimiento esperable por unidad de esfuerzo.

Glosario procedente del material docente. La extracción automática por frecuencia produce vocabulario corriente que el modelo ya reconoce. Un glosario elaborado a partir del temario o las transparencias tendría el contenido que la técnica necesita, y permitiría reevaluar el prompting en condiciones justas.

Métricas de comprensión. Las métricas propuestas miden la supervivencia de unidades léxicas concretas. El paso siguiente sería medir si el alumno entiende lo mismo leyendo la transcripción que escuchando la clase, lo que exige evaluación con personas.

Evaluación con alumnado que está aprendiendo la lengua. Es la continuación con mejor relación entre coste y aportación, por tres motivos. La población es grande: más de un millón de alumnos de nacionalidad extranjera en el sistema educativo español no universitario. La evidencia de que el subtítulo le beneficia está establecida sobre vídeo grabado (Montero Perez y cols., 2013; Gernsbacher, 2015) pero apenas explorada en directo; y el único estudio localizado en ese escenario (Q. Wang y Chen, 2023) deja abierta justo la pregunta que este sistema permitiría atacar, porque midió participación y calificaciones pero no calidad de la transcripción. Un sistema que además *mide* qué negaciones y qué terminología sobreviven permitiría relacionar por primera vez ambas cosas: cuánta degradación tolera quien está aprendiendo la lengua antes de descolgarse de la clase. Es la hipótesis que la sección 2.1.2 deja planteada y que este trabajo no puede responder.

Adaptación al español de una arquitectura de transductor. El contraste con un transductor multilingüe dejó una vía abierta: el candidato conservaba mucho mejor el contenido crítico y corría entre cinco y nueve veces más rápido, y quedó descartado por un motivo concreto y potencialmente reparable, la deriva al inglés. Un ajuste fino sobre español, o una variante que permita imponer el idioma, cambiarían el balance por completo. Es la continuación con mejor relación entre esfuerzo y recorrido, porque parte de un sistema que ya gana en las métricas que importan.

Repetir la comparación de arquitecturas sobre el corpus objetivo. La diferencia entre habla leída y espontánea fue lo bastante grande como para desaconsejar extrapolar: lo medido sobre corpus sustitutos no permite anticipar qué ocurriría sobre clases reales.

Referencias

- AENOR. (2012). *UNE 153010:2012. subtitulado para personas sordas y personas con discapacidad auditiva*. Asociación Española de Normalización y Certificación. (Fija criterios de velocidad de lectura, segmentacion y presentacion)
- AMD. (2025). *Use ROCm on Radeon and Ryzen: WSL how-to guide*. https://rocm.docs.amd.com/projects/radeon-ryzen/en/latest/docs/install/installrad/wsl/howto_wsl.html. (Soporte de produccion sobre el sub-sistema de Windows via ROCDXG, tambien a traves de `/dev/dxg`)
- Ardila, R., Branson, M., Davis, K., Kohler, M., Meyer, J., Henretty, M., ... Weber, G. (2020). Common Voice: A massively-multilingual speech corpus. En *Proceedings of the 12th language resources and evaluation conference (lrec)* (pp. 4218–4222). Marsella, Francia.
- Bisani, M., y Ney, H. (2004). Bootstrap estimates for confidence intervals in ASR performance evaluation. En *Ieee international conference on acoustics, speech, and signal processing (icassp)* (Vol. 1, pp. 409–412). (Fundamenta el remuestreo por fragmentos y no por palabras)
- Conneau, A., Ma, M., Khanuja, S., Zhang, Y., Axelrod, V., Dalmia, S., ... Bapna, A. (2022). FLEURS: Few-shot learning evaluation of universal representations of speech. *arXiv preprint arXiv:2205.12446*.
- del Agua, M. A., Giménez, A., Serrano, N., Andrés-Ferrer, J., Civera, J., Sanchis, A., y Juan, A. (2014). The transLectures-UPV toolkit. En *Advances in speech and language technologies for iberian languages (iberspeech)* (Vol. 8854, pp. 269–278). Las Palmas de Gran Canaria: Springer. (Describe el corpus poliMedia, con mas de 115 horas de clases en espanol transcritas manualmente)
- Efron, B., y Tibshirani, R. J. (1993). *An introduction to the bootstrap*. New York: Chapman & Hall.
- Gernsbacher, M. A. (2015). Video captions benefit everyone. *Policy Insights from the Behavioral and Brain Sciences*, 2(1), 195–202. (Revision de mas de cien estudios empiricos: el subtitulado en la misma lengua mejora comprension, atencion y memoria, y beneficia especialmente a quien ve el material en una lengua que no es la suya)
- Graves, A. (2012). Sequence transduction with recurrent neural networks. En *Icml*

- workshop on representation learning*. (Preprint disponible en arXiv:1211.3711)
- Gulati, A., Qin, J., Chiu, C.-C., Parmar, N., Zhang, Y., Yu, J., . . . Pang, R. (2020). Conformer: Convolution-augmented transformer for speech recognition. En *Proceedings of interspeech* (pp. 5036–5040).
- Hernández Mena, C. D., y Herrera Camacho, A. (2014). CIEMPIESS: A new open-sourced Mexican Spanish radio corpus. En *Proceedings of the 9th international conference on language resources and evaluation (lrec)* (pp. 371–375). Reikiavik, Islandia. (Sus referencias omiten sistemáticamente la acentuación; ver R11)
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., . . . Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- Jefatura del Estado. (2013). *Real decreto legislativo 1/2013, de 29 de noviembre, por el que se aprueba el texto refundido de la ley general de derechos de las personas con discapacidad y de su inclusión social*. Boletín Oficial del Estado, núm. 289, de 3 de diciembre de 2013.
- Kafle, S., y Huenerfauth, M. (2017). Evaluating the usability of automatically generated captions for people who are deaf or hard of hearing. En *Proceedings of the 19th international acm sigaccess conference on computers and accessibility (assets)* (pp. 165–174). Baltimore, MD: ACM. (Evidencia experimental de que el WER predice mal la utilidad percibida de un subtítulo automático)
- Kim, S., Arora, A., Le, D., Yeh, C.-F., Fuegen, C., Kalinli, O., y Seltzer, M. L. (2021). Semantic distance: A new metric for ASR performance analysis towards spoken language understanding. En *Proceedings of interspeech* (pp. 1977–1981). (Preprint disponible en arXiv:2104.02138)
- Kolobov, R., Okhapkina, O., Omelchishina, O., Platunov, A., Bedyakin, R., Moshkin, V., . . . Mikhaylovskiy, N. (2021). MediaSpeech: Multilanguage ASR benchmark and dataset. *arXiv preprint arXiv:2103.16193*.
- Linder, K. (2016). *Student uses and perceptions of closed captions and transcripts: Results from a national study* (Inf. Téc.). Corvallis, OR: Oregon State University Ecampus Research Unit. (Encuesta a 2.124 estudiantes de 15 universidades; el 98,6 % considera útiles los subtítulos y el 75 % los usa como apoyo al aprendizaje)
- Macháček, D., Dabre, R., y Bojar, O. (2023). Turning whisper into real-time transcription system. En *Proceedings of the 13th international joint conference on natural language processing and the 3rd conference of the asia-pacific chapter of the association for*

- computational linguistics: System demonstrations* (pp. 17–24). Bali, Indonesia: Association for Computational Linguistics. (Preprint disponible en arXiv:2307.14743)
- Microsoft. (2024). *Enable NVIDIA CUDA on WSL 2*. <https://learn.microsoft.com/windows/ai/directml/gpu-cuda-in-wsl>. (La tarjeta se expone dentro del subsistema a través de `/dev/dxg`, no de los dispositivos de un Linux nativo)
- Ministerio de Educación, Formación Profesional y Deportes. (2024). *Estadística de las enseñanzas no universitarias. datos avance del curso 2023-2024*. Subdirección General de Estadística y Estudios, Gobierno de España. (1.066.875 alumnos de nacionalidad extranjera, el 12,2 % del total en enseñanzas de Régimen General no universitarias)
- Montero Perez, M., Van Den Noortgate, W., y Desmet, P. (2013). Captioned video for L2 listening and vocabulary learning: A meta-analysis. *System*, 41(3), 720–739. (Metaanálisis de 18 estudios; efecto grande sobre comprensión oral y sobre adquisición de vocabulario ($g = 0,87$))
- Organización de las Naciones Unidas. (2006). *Convención sobre los derechos de las personas con discapacidad*. Resolución A/RES/61/106. Artículo 24, educación. (Ratificada por España en 2008)
- Parlamento Europeo y Consejo de la Unión Europea. (2016). *Reglamento (ue) 2016/679, general de protección de datos*. Diario Oficial de la Unión Europea, L 119.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., y Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. En *Proceedings of the 40th international conference on machine learning* (Vol. 202). (Preprint disponible en arXiv:2212.04356)
- Rekesh, D., Koluguri, N. R., Kriman, S., Majumdar, S., Noroozi, V., Huang, H., ... Ginsburg, B. (2023). Fast Conformer with linearly scalable attention for efficient speech recognition. En *Ieee automatic speech recognition and understanding workshop (asru)*. (Preprint disponible en arXiv:2305.05084)
- Silero Team. (2021). *Silero VAD: pre-trained enterprise-grade voice activity detector*. <https://github.com/snakers4/silero-vad>. (Alternativa entrenada al detector de energía empleado en este trabajo)
- Vanderplank, R. (1988). The value of teletext sub-titles in language learning. *ELT Journal*, 42(4), 272–281. (Trabajo fundacional sobre subtítulo en la misma lengua como apoyo al aprendizaje de idiomas)
- Wang, C., Rivière, M., Lee, A., Wu, A., Talnikar, C., Haziza, D., ... Dupoux, E. (2021).

- VoxPopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation. En *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing*. Association for Computational Linguistics.
- Wang, Q., y Chen, Y. (2023). The role of live transcripts in synchronous online L2 classrooms: Learning outcomes and learner perceptions. *Education and Information Technologies*. (El unico estudio localizado sobre transcripcion EN VIVO y no sobre video grabado: no mejora las calificaciones, pero el alumnado de menor competencia completa un 24,6 % mas de actividades de clase. DOI 10.1007/s10639-023-11784-8)
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., . . . Brew, J. (2020). Transformers: State-of-the-art natural language processing. En *Proceedings of the 2020 conference on empirical methods in natural language processing: System demonstrations*. Association for Computational Linguistics.
- Xu, H., Jia, F., Majumdar, S., Huang, H., Watanabe, S., y Ginsburg, B. (2023). Efficient sequence transduction by jointly predicting tokens and durations. En *Proceedings of the 40th international conference on machine learning* (Vol. 202). (Define la variante TDT del transductor empleada por el modelo contrastado en exp-004. Preprint en arXiv:2304.06795)

A. Reproducibilidad

Este apéndice recoge lo necesario para reproducir cualquier cifra de la memoria. No es documentación de instalación: es la respuesta a la pregunta de si los resultados presentados pueden volver a obtenerse y con qué grado de fidelidad.

A.1. Entorno de ejecución

Los experimentos se ejecutaron sobre una tarjeta gráfica AMD con la pila de cómputo ROCm, en un sistema Linux. Esta elección tiene una consecuencia que conviene declarar: parte del software habitual del campo asume tarjetas de otro fabricante, y algunas implementaciones optimizadas para inferencia no estaban disponibles. La comparativa se realizó por tanto sobre la implementación de referencia de la biblioteca *Transformers* (Wolf y cols., 2020), más lenta que las alternativas especializadas pero disponible por igual en ambas plataformas, lo que evita que la elección de hardware se confunda con una diferencia de resultados.

El intérprete de Python se creó reutilizando los paquetes del sistema de forma deliberada, porque reinstalar la biblioteca de cálculo dentro del entorno aislado rompe el soporte de la tarjeta gráfica. Es un detalle menor que causa horas de depuración si se desconoce.

A.2. Trazabilidad de los resultados

El procedimiento actual hace que cada ejecución produzca un fichero de resultados que registra, junto a las métricas:

- El identificador del commit del código que la produjo.
- Si el árbol de trabajo tenía cambios sin confirmar en ese momento, lo que indica que el commit registrado no basta para reconstruir el estado exacto.
- El resumen criptográfico del manifiesto de corpus empleado, junto con su número de fragmentos, y el de la submuestra efectiva cuando el experimento evalúa solo una parte de él.
- La configuración completa: modelo, estrategia de decodificación, semilla, parámetros del experimento y versiones de las bibliotecas relevantes.

El tercer punto se incorporó tras un incidente. Al ampliar un corpus se sobrescribió su manifiesto, y los resultados anteriores quedaron apuntando a un fichero cuyo contenido ya era distinto sin que nada lo advirtiera. El resumen criptográfico permite detectar esa discrepancia; posteriormente se añadió también la posibilidad de que varias muestras de la misma fuente coexistan, que la previene.

Hasta dónde llega, en la práctica

Decir «cada ejecución registra» describe el procedimiento, no el archivo: el mecanismo nació de aquel incidente, de modo que alcanza en distinto grado a lo ejecutado antes y después. Conviene dar las cifras, porque presentar la trazabilidad como una propiedad uniforme sería justamente el tipo de afirmación de la que este trabajo dedica un capítulo a desconfiar. Sobre los cincuenta y dos ficheros de resultados que sostienen esta memoria:

- Todos registran su configuración completa, que es lo que permite repetir la ejecución, y todos registran un identificador de commit.
- En cuarenta y cuatro ese identificador vale `sin-git`, porque se produjeron antes de que el trabajo estuviera bajo control de versiones. Son los más antiguos, y entre ellos está el barrido de línea base.
- Once registran el resumen criptográfico del manifiesto, que es la parte que se añadió tras el incidente. En los once, recalcularlo hoy devuelve el valor que el fichero declara.
- Uno solo se produjo con el árbol de trabajo limpio. El resto de los que lo declaran tenían cambios sin confirmar, cosa que el propio bloque advierte y que significa que el commit registrado no basta para reconstruir el estado exacto.

El programa `tools/auditar_resultados.py` recorre el archivo entero y falla si algo no cuadra. Aplica dos comprobaciones, según lo que cada fichero permita.

La directa consiste en *recalcular* el resumen criptográfico del manifiesto y compararlo con el que el resultado declara. Merece subrayarse por lo que tiene de lección repetida: guardar el resumen no detecta nada por sí solo, y durante un tiempo nadie lo recalculó, de modo que la salvaguarda contra el incidente era exactamente la misma clase de fallo que este trabajo documenta en otros sitios, un dato declarado que ninguna comprobación consulta. Cuando el experimento evalúa solo una parte del manifiesto se compara además

el resumen de esa submuestra, porque el del fichero completo identifica material que no se ha medido.

La indirecta cubre a los resultados anteriores al mecanismo, que no llevan resumen: el número de fragmentos y de palabras de referencia que declara cada fichero tiene que coincidir con el que produce el manifiesto de hoy. Hoy señala cinco ficheros, y son precisamente los del incidente: mediciones sobre cuarenta fragmentos de un corpus que después se amplió a mil doscientos. Ese es el motivo de que la tabla 5.2 marque esa fila, y de que las comparaciones entre modelos deban hacerse sobre las tablas por corpus.

Dos categorías quedan fuera de la comprobación indirecta sin que eso sea un hueco. Los barridos de ingeniería vuelven a trocear el audio en ventanas o segmentos, así que no declaran un recuento de fragmentos con el que contrastar; llevan resumen criptográfico, que es la comprobación fuerte. Y los experimentos que emplean glosario reservan una fracción de los fragmentos para extraerlo y evalúan sobre el resto, de modo que su recuento queda por debajo del manifiesto a propósito: el programa descuenta esa reserva, porque denunciar como deriva la propia salvaguarda contra la fuga de información habría enterrado los cinco casos reales bajo cuatro falsos.

Merece una advertencia de uso, porque costó tiempo: el recuento de palabras que guardan los resultados es el de las referencias *ya normalizadas*, y el normalizador convierte los signos en espacios, de modo que un corpus puede declarar más palabras de las que tiene en crudo. Compararlo contra el texto sin normalizar produce discrepancias que no existen.

Qué se versiona y qué no

Los manifiestos de corpus, los programas y las tablas y figuras generadas están en el repositorio. No lo están, deliberadamente, tres cosas: el audio, porque su licencia no permite redistribuirlo; los ficheros de resultados en crudo, porque son la salida de los programas y no su entrada; y el adaptador entrenado, por tamaño.

La consecuencia hay que decirla sin adornos: **regenerar cualquier cifra con un solo comando es cierto sobre una instalación que ya tenga el corpus descargado**, que es el supuesto en el que se ha trabajado. Un tercero que parta del repositorio necesita antes obtener el audio de sus fuentes originales mediante los programas de descarga incluidos, disponer de una tarjeta gráfica, y reentrenar el adaptador para reproducir el resultado del ajuste fino. Los manifiestos hacen esa reconstrucción posible y verificable, que es su función; no la hacen inmediata.

A.3. Identidad de un resultado

Los ficheros de resultados se nombran según el modelo, el corpus y la estrategia de decodificación empleados. Esta última forma parte del nombre porque se comprobó que dos ejecuciones idénticas salvo por ese parámetro producían tasas de error que diferían en más de doce puntos. Un resultado identificado únicamente por modelo y corpus habría permitido comparar cifras no comparables.

La regla general que se derivó: cualquier eje de variación que se introduzca debe incorporarse al nombre del resultado, no solo a su contenido.

A.4. Generación de tablas y figuras

Ninguna cifra de esta memoria se ha transcrito manualmente. Las tablas y figuras se generan mediante programas que leen los ficheros de resultados y producen directamente el código LaTeX o el gráfico correspondiente, que la memoria incorpora por inclusión.

La consecuencia práctica es que regenerar los informes tras una nueva ejecución actualiza la memoria sin intervención. La consecuencia metodológica es más relevante: una cifra copiada a mano deja de estar vinculada a la ejecución que la produjo en el momento en que se copia, y no hay forma de detectar la divergencia posterior.

A.5. Verificaciones previas

Tres comprobaciones se ejecutan antes de dar por válido un resultado, y las tres surgieron de errores que no producían ningún fallo visible.

Verificación de la tarjeta gráfica. No basta con comprobar que el sistema la detecta: se verifica que calcula correctamente y que completa un ciclo de entrenamiento con descenso efectivo de la función de pérdida.

Verificación del entrenamiento. Antes de evaluar un modelo ajustado se comprueba que la función de pérdida se encuentra en el rango esperado y que desciende entre épocas. Un ajuste con las etiquetas mal alineadas termina sin errores, guarda su resultado y produce métricas evaluables; solo el valor absoluto de la pérdida lo delata.

Verificación del procesamiento por lotes. Agrupar fragmentos acelera sustancialmente los barridos, pero solo es admisible si no altera las transcripciones. Se comprueba

comparándolas con las obtenidas de una en una. Una optimización que cambia los resultados no es una optimización: es un factor de confusión.

A.6. Estructura del repositorio

El trabajo separa estrictamente el material de investigación del de la aplicación; ambos no comparten código y se comunican mediante una interfaz documentada. Cada experimento reside en su propia carpeta, con el programa que lo ejecuta, una descripción de su hipótesis y su diseño, y sus resultados.

Los manifiestos de corpus se versionan; el audio no. Un manifiesto contiene las rutas, las transcripciones de referencia, las duraciones y los metadatos disponibles, de modo que el material puede reconstruirse desde su fuente original sin almacenar los ficheros de sonido.

B. Registro de riesgos

Este apéndice recoge el registro de riesgos que guió la ejecución del trabajo. Se incluye por dos motivos. El primero es que varias decisiones de diseño de los capítulos anteriores no se entienden sin él: no se eligieron por ser las mejores en abstracto, sino por ser las que sobrevivían a un riesgo concreto. El segundo es que seis de los dieciséis riesgos *no* estaban previstos: se detectaron midiendo, no planificando, y esa es en sí misma una observación sobre cómo se planifica un trabajo de esta naturaleza.

B.1. Riesgos previstos en la planificación inicial

Los diez de la tabla B.1 se identificaron antes de empezar a medir. Dos de ellos se materializaron, y que estuvieran previstos es lo que evitó que descarrilaran el trabajo.

R1 se materializó en su forma más costosa: el corpus objetivo no se obtuvo, y el trabajo se ejecutó sobre sustitutos. R6 también, y de ahí que dos de las tres técnicas evaluadas no produzcan efecto concluyente. Que ambos estuvieran previstos es lo que permitió que su materialización no descarrilara el trabajo: el criterio de significación y la política de marcar como no concluyente lo que no alcanza potencia estaban fijados antes de saber qué iban a dar los datos.

B.2. Riesgos detectados midiendo

Los seis siguientes no aparecían en la planificación. Se documentan aparte porque comparten un rasgo, visible en la tabla B.2: ninguno se manifestó como un problema, sino como un resultado creíble.

R14 y R16 merecen un comentario conjunto, porque ilustran los dos extremos del mismo problema. R14 era invisible desde dentro: dos ejecuciones del mismo modelo sobre el mismo corpus daban cifras incomparables y nada lo delataba. R16 era invisible desde fuera: los resultados seguían siendo correctos, y lo que envejecía era su premisa. Ninguno de los dos habría aparecido en una revisión del código o de los resultados; el primero exigió comparar ejecuciones que no debían diferir, y el segundo mirar fuera de la familia de modelos elegida.

R15 condicionó el método de trabajo más que ningún otro. La norma adoptada ante cualquier decisión de alcance que no pudiera cerrarse en el momento fue la misma: elegir

una opción razonable por defecto, dejar constancia escrita del motivo en un registro de decisiones y marcarla como revisable, en lugar de detener el trabajo. El coste de esa política es que varias decisiones quedan documentadas y argumentadas pero pendientes de validación; el beneficio es que ninguna de ellas bloqueó el avance del resto.

Id	Riesgo	Mitigación aplicada	Estado
R1	No existe corpus educativo en español con transcripción de referencia fiable y de acceso abierto	Dos planes aplicados: corpus públicos con el desajuste de dominio medido y declarado, y subtítulos existentes verificados a mano. El tercero, la solicitud institucional del corpus objetivo, queda redactado y dirigido pero su tramitación excede el plazo	Materializado
R2	Entrenar sobre tarjeta gráfica AMD puede resultar inviable	Verificar en la primera fase que la tarjeta <i>entrena</i> , con un ciclo mínimo, antes de comprometer la técnica que lo necesita	Mitigado
R3	Datos insuficientes para el ajuste fino	Última en el orden de ejecución y segunda en el de recorte; no comprometerla hasta resolver R1 y R2	No materializado
R4	Latencia insuficiente para operar en directo	Instrumentar la medida desde el primer prototipo y reportar la latencia real aunque no cumpla	Mitigado
R5	El doble núcleo duplica el trabajo	Tratar la aplicación como vehículo de demostración y banco de pruebas, no como contribución con validación propia	Materializado
R6	La comparativa no arroja diferencias concluyentes	Protocolo congelado de antemano para que un resultado negativo siga siendo defendible; reportar variabilidad y no solo medias	Materializado
R7	Redacción tardía	Escribir desde la primera fase, en borradores parciales	Mitigado
R8	Resultados no reproducibles	Una configuración por experimento, semillas fijadas, procedencia en cada salida, tablas generadas y no copiadas	Mitigado
R9	Tratamiento de datos personales	No grabar material propio; el corpus empleado tiene licencia de investigación. La restricción de red la impone la aplicación	Mitigado
R10	Disponibilidad personal	Planificación en fases relativas y puntos de recorte explícitos	Mitigado

Tabla B.1: Riesgos identificados durante la planificación.

Id	Riesgo	Mitigación aplicada	Estado
R11	La calidad de la transcripción de referencia varía entre corpus y se confunde con rendimiento del modelo	Auditoría manual de una muestra antes de adoptar cualquier corpus; reportar el error con y sin normalización de tildes como cota superior e inferior	Mitigado
R12	Efecto suelo: sobre voz leída el modelo sin adaptar ya acierta casi todo y no queda margen donde las técnicas puedan demostrar nada	Trasladar la comparativa al material de habla espontánea, donde el error base deja margen	Mitigado
R13	Desajuste de variedad dialectal entre corpus y destino	Medido: a registro equivalente la diferencia entre variedades es de dos décimas, frente a los veinte puntos que separa el registro. Queda descartado como factor	Descartado
R14	La configuración de decodificación es un factor de confusión que mueve el error hasta trece puntos	Congelarla e incorporarla a la identidad de cada resultado	Mitigado
R15	Decisiones de alcance bloqueadas a la espera de validación externa	Tomar decisiones provisionales documentadas y marcadas como revisables, con cola viva de preguntas pendientes; avanzar mientras tanto en lo que no depende de terceros	Materializado
R16	El modelo base envejece durante el desarrollo y la elección deja de estar justificada	Medirlo en lugar de argumentarlo: contraste con una arquitectura distinta sobre 800 fragmentos, y catálogo de modelos auditados que se revisa igual que el de corpus	Mitigado

Tabla B.2: Riesgos detectados durante la ejecución, ausentes de la planificación inicial.

C. Despliegue y uso del sistema

El capítulo 5 explica *por qué* el despliegue tiene la forma que tiene. Este apéndice explica *cómo* se ejecuta, paso a paso, y está escrito para quien tenga que ponerlo en marcha en un centro sin haber participado en el desarrollo: el responsable informático el primer día, y el docente cada mañana.

C.1. Qué hace falta antes de empezar

El sistema se despliega en **un solo equipo del centro**, que actúa de servidor y hace todo el trabajo. Los dispositivos del alumnado no necesitan absolutamente nada salvo un navegador y estar en la misma red: no instalan, no calculan y no conceden permisos. Esa asimetría es deliberada y se justifica en la sección 5.6.1; para el despliegue lo que implica es que solo hay un equipo del que preocuparse.

Los requisitos de ese equipo no son una lista de recomendaciones: cada uno procede de una medición del capítulo 5 y puede comprobarse antes de dar una clase con el sistema. Se presentan en dos tablas porque responden a dos preguntas distintas: la tabla C.1 dice *qué equipo hace falta*, que es una decisión de compra, y la tabla C.2 *qué hay que preparar* en el que ya se tenga.

Conviene señalar lo que *no* aparece en ninguna de las dos, porque suele darse por supuesto: no hace falta conexión a internet durante la clase, ni cuenta en ningún servicio, ni licencia de software. La única descarga es la del modelo, y ocurre una sola vez en el primer arranque. Esto es consecuencia directa de que el reconocimiento se ejecute en el centro y no en la nube (sección 3.1.3).

De dónde salen esas cifras

Los tres requisitos de memoria y almacenamiento proceden de una medición directa del servicio en ejecución, no del tamaño nominal del modelo, que es lo que suele citarse y además induce a error. La tabla C.3 recoge el detalle.

Elemento	Requisito	De dónde sale
Tarjeta gráfica	Dedicada, con al menos 4 GB de memoria propia. No hace falta gama alta.	Medido: el modelo ocupa 1,60 GB de memoria de vídeo en el peor caso (tabla C.3). El resto es margen para el escritorio del propio equipo y la fragmentación de memoria. Sin tarjeta dedicada el sistema arranca, pero no sostiene el ritmo del audio en directo (sección 5.6.5).
Memoria del sistema	8 GB.	El proceso de reconocimiento ocupa 2,76 GB residentes; la aplicación y el sistema operativo caben holgadamente en el resto.
Procesador	Cuatro núcleos o más. Cualquiera de los últimos años sirve.	No calcula el modelo: despacha el trabajo a la tarjeta gráfica. Conviene saber que un núcleo queda ocupado al 100 % de forma permanente mientras el servicio está vivo, incluso en reposo: es espera activa del entorno de ejecución de la GPU, no trabajo real, y no indica ningún problema.
Almacenamiento	Depende de la vía: 5 GB nativa, 100 GB en contenedores sobre AMD.	El modelo son 2,85 GB en ambos casos. La diferencia no la causa el sistema sino la imagen base de la plataforma de AMD, y se detalla más abajo porque es lo bastante grande como para condicionar la elección de equipo.

Tabla C.1: Requisitos de hardware del equipo servidor, con la medición que respalda cada uno.

Modelo	Disco (GB)	VRAM pico (GB)	RAM del proceso (GB)
whisper-small	0.90	0.56	2.57
whisper-medium	2.85	1.60	2.76
whisper-medium + LoRA	2.85	1.63	2.78

Tabla C.3: Huella de recursos del servicio de reconocimiento, medida sobre AMD Radeon RX 9070 XT con precisión `float16` y una ventana completa de 30 s de audio, que es el caso peor. El pico de VRAM mayor observado es 1.63 GB, de donde sale el requisito de 4 GB de la tabla C.1: el margen cubre el gestor de ventanas del propio equipo y la fragmentación de memoria.

Dos lecturas de esa tabla importan para decidir la compra. La primera es que el requi-

Elemento	Requisito	Observación
Sistema operativo	Linux permite el despliegue en contenedores; en macOS con Apple Silicon hay que ejecutar de forma nativa.	No es indiferente: en macOS los contenedores no alcanzan la GPU. Conviene leer la sección C.2 <i>antes</i> de elegir procedimiento.
Acceso a la GPU	Con AMD, pertenecer a los grupos <code>video</code> y <code>render</code> . Con NVIDIA, el conjunto de herramientas de contenedores del fabricante.	Es el fallo de arranque más frecuente. Se comprueba con <code>getent group video render</code> , cuyos identificadores numéricos deben coincidir con los del descriptor de contenedores.
Micrófono	De solapa o diadema, próximo a la boca del docente. No vale el micrófono integrado del equipo ni uno ambiental.	Medido: el sistema tolera ruido hasta 15 dB de relación señal-ruido, y por debajo se degrada de golpe en lugar de gradualmente (sección 5.6.6). La proximidad del micrófono es lo que mantiene ese margen cuando el aula se anima.
Red	Red local del centro, con el equipo servidor y los dispositivos del alumnado en el mismo rango.	La aplicación rechaza toda conexión que no provenga de un rango privado. No es configurable: es un requisito legal del sistema (sección 3.1.3). Si hiciera falta acceso desde fuera, la vía es una red privada virtual hasta el centro.
Navegador	Cualquiera moderno, tanto en el equipo del docente como en los del alumnado.	La emisión debe abrirse en el propio equipo servidor, por una restricción de los navegadores que se explica en el paso 4.

Tabla C.2: Requisitos de entorno y periféricos.

sito de memoria de vídeo es **mucho menor de lo que sugiere el tamaño del modelo en disco**: 2,85 GB ocupados en almacenamiento se traducen en 1,60 GB de memoria de vídeo, porque los pesos se cargan en precisión reducida. Una tarjeta de 4 GB, que hoy es la gama de entrada, sobra. Lo que el sistema necesita no es una tarjeta grande sino, simplemente, una tarjeta.

La segunda es que existe una salida si el equipamiento disponible se queda corto: el modelo inmediatamente menor baja el pico a 0,56 GB y el almacenamiento a 0,90 GB. Cuesta calidad, entre uno y dos puntos de tasa de error según el material (tabla 5.3),

pero es una degradación conocida y medida, no una sorpresa. La diferencia entre ambos modelos se estrecha además precisamente en habla espontánea, que es el registro del aula.

Incorporar el modelo adaptado no cambia el cuadro: el adaptador añade 19 MB en disco y 30 MB de memoria de vídeo, porque no sustituye al modelo sino que le agrega unas matrices pequeñas. Es la ventaja práctica de la técnica que ganó la comparativa, y la razón de que pueda distribuirse por separado.

El almacenamiento, que es la sorpresa desagradable

Las cifras anteriores se refieren al modelo. El despliegue completo en contenedores sobre AMD ocupa mucho más, y conviene decirlo con claridad porque es el requisito que más se subestima:

Componente	Disco
Imagen del reconocimiento (AMD)	83,5 GB
Caché de núcleos de cálculo compilados	11,6 GB
Caché del modelo	3,1 GB
Imagen de la aplicación	0,35 GB
Total aproximado	~100 GB

Tabla C.4: Ocupación en disco del despliegue en contenedores sobre AMD, medida con `docker system df -v` en el equipo de desarrollo.

Lo llamativo es de dónde viene: **no es el modelo**, que son 2,85 GB, sino la imagen base de la plataforma de cálculo de AMD. Inspeccionarla por dentro explica el motivo, y el motivo es que el contenedor transporta código para tarjetas que el equipo no tiene.

Esa plataforma compila por anticipado los núcleos de cálculo para cada arquitectura de tarjeta, y la imagen empleada los incluye *todos*: su etiqueta declara literalmente `device=all`. Medido dentro de ella, una sola de sus bibliotecas de álgebra contiene núcleos para **veinticuatro arquitecturas distintas**, desde las de centro de datos hasta tres generaciones de tarjetas de consumo; de sus 792 MB, los que corresponden a la tarjeta empleada en este trabajo son **10 MB**. Otro componente aparece como trece bibliotecas separadas, una por arquitectura, que suman 4,4 GB y de las que se usa una.

A ello se añaden dos circunstancias más. La imagen incluye el kit de *desarrollo* de la plataforma, 15 GB de compiladores y cabeceras que no hacen falta para ejecutar inferencia pese a que la imagen se anuncia como de ejecución. Y los 11,6 GB de núcleos compilados en el primer arranque se generan porque, para la generación de tarjeta concreta empleada,

esa plataforma no distribuye binarios precompilados y hay que producirlos en destino.

Conviene precisar el grado de confianza de las cifras. Las de la tabla proceden de **docker system df -v**, que es lo que el sistema informa; el contenido real del sistema de ficheros de la imagen, medido desde dentro, son 34 GiB. La diferencia se debe a cómo el motor de contenedores contabiliza capas almacenadas y desplegadas, y no se ha podido acotar con precisión por requerir privilegios de administrador sobre su directorio de datos. Para planificar un despliegue conviene por tanto tomar la cifra alta, que es la conservadora.

Tres consecuencias prácticas. La primera, que un equipo con un disco pequeño, que es lo habitual en un portátil de centro, puede quedarse sin espacio por esta vía y no por el modelo. La segunda, que la ejecución nativa ocupa alrededor de 5 GB y evita el problema por completo, lo que la convierte en una alternativa razonable y no en un apaño. La tercera, que esto es reducible: partir de una imagen base restringida a la familia de tarjeta del equipo, en lugar de a todas, eliminaría por construcción la mayor parte de lo que se transporta. No se ha hecho, de modo que no se aporta la cifra resultante; queda como la mejora pendiente más rentable del despliegue.

Hay una última observación que afecta menos al espacio y más a la reproducibilidad, y por eso conviene dejarla escrita: la imagen base empleada es una compilación **nocturna** y no una versión estable, según declaran sus propios parámetros de construcción. Para el desarrollo es aceptable, porque era la vía de tener soporte de una tarjeta muy reciente; para un despliegue en un centro no lo es, porque la referencia se mueve bajo los pies y dos construcciones separadas en el tiempo no producen el mismo sistema. Fijar una versión estable concreta es requisito previo a cualquier distribución, y es coherente con el criterio que gobierna el resto del trabajo: un resultado que no se puede reconstruir no es un resultado.

Del perfil de NVIDIA no se da cifra porque no se ha medido: no se dispuso de una tarjeta de ese fabricante. Cabe esperar que sea sustancialmente menor, porque su imagen base es de ejecución y no de desarrollo, pero es una expectativa y no un dato.

C.2. Cómo se distribuye el sistema

Qué es un contenedor, y por qué se usa aquí

El sistema se entrega **contenerizado**. Un contenedor es un paquete que incluye el programa junto con todas sus dependencias, es decir el intérprete, las bibliotecas y sus

versiones exactas, de modo que se ejecuta igual en cualquier equipo que tenga el motor de contenedores instalado, sin instalar nada más en el sistema anfitrión. Docker es la implementación empleada aquí, y es la habitual.

La razón de recurrir a ello no es seguir una moda de despliegue: es que el entorno de este sistema es especialmente incómodo de montar a mano. Requiere una versión concreta de la biblioteca de cálculo, compilada contra la plataforma del fabricante de la tarjeta gráfica, más el entorno de ejecución de la aplicación. Reproducir eso en el equipo de un centro sería una tarde de trabajo para alguien con conocimientos, y una fuente permanente de averías al actualizar el sistema operativo. Con contenedores, el centro instala Docker y ejecuta un comando.

El sistema se reparte en **dos contenedores** y no en uno, porque tienen necesidades distintas: el del reconocimiento necesita acceso a la tarjeta gráfica y pesa varios gigabytes; el de la aplicación web no necesita tarjeta y es pequeño. Separarlos permite además sustituir el modelo sin tocar la aplicación, que es la frontera descrita en la sección 5.6.12.

Sobre qué sistemas operativos se puede ejecutar

Sistema	Vía	Acelerador	Estado
Linux, tarjeta AMD	Contenedores	ROCm	<code>make docker-amd</code> . Es la plataforma de desarrollo del trabajo.
Linux, tarjeta NVIDIA	Contenedores	CUDA	<code>make docker-nvidia</code> . Requiere el conjunto de herramientas de contenedores del fabricante.
macOS, Apple Silicon	Nativa	MPS	<code>make demo</code> . En contenedor no alcanza la GPU; se explica más abajo.
Windows, como cliente	Funciona sin nada especial. Es solo un navegador. Es además el caso más frecuente: los dispositivos del alumnado.		
Windows, como servidor	No probado. Existe una vía a través del subsistema Linux de Windows, pero el descriptor de contenedores requiere cambios; se detalla más abajo.		
Sin tarjeta gráfica	Arranca y transcribe, pero no sostiene el directo. Sirve para verificar la instalación y la red del aula, no para dar clase.		

Tabla C.5: Sistemas operativos sobre los que puede ejecutarse el servidor.

De dónde salen las imágenes

Conviene ser preciso en esto, porque determina lo que un centro tiene que hacer y lo que todavía no puede hacer.

Las imágenes de este sistema no están publicadas en ningún registro público. No existe, por tanto, una orden `docker pull` que las descargue: se construyen en el propio equipo a partir del repositorio del proyecto, y eso es lo que hace la opción `--build` de los comandos de la tabla C.5. El proceso, la primera vez, consiste en descargar las imágenes base públicas de las que se parte, instalar sobre ellas las dependencias declaradas y copiar el código del proyecto.

Las imágenes base sí proceden de registros públicos, y son las que recoge la tabla C.6:

Componente	Imagen base
Reconocimiento, AMD	<code>rocm/pytorch</code>
Reconocimiento, NVIDIA	<code>pytorch/pytorch:2.5.1-cuda12.4-cudnn9-runtime</code>
Aplicación, compilación	<code>mcr.microsoft.com/dotnet/sdk:10.0</code>
Aplicación, ejecución	<code>mcr.microsoft.com/dotnet/aspnet:10.0</code>

Tabla C.6: Imágenes base de las que parten los contenedores del sistema.

La versión concreta de la imagen base del reconocimiento se fija desde fuera y no dentro del fichero de construcción, a propósito: cada combinación de tarjeta y controlador exige una distinta, y codificarla obligaría a editar el fichero en cada despliegue.

Qué implica que no estén publicadas. Hoy, el centro necesita el repositorio del proyecto y una máquina capaz de construir, lo que incluye descargar las dependencias de compilación de la aplicación. Publicar las imágenes en un registro, por ejemplo el del propio alojamiento del código, reduciría el despliegue a descargar el descriptor y ejecutar `docker compose up`, sin repositorio ni construcción, y haría el primer arranque mucho más rápido y mucho menos sensible al equipo. Es una mejora de distribución evidente y queda como tarea pendiente, no como decisión: no se ha hecho porque el trabajo no ha llegado a la fase de distribuir el sistema a terceros.

El caso de Windows

Merece tratamiento propio por una razón práctica: es, con diferencia, el sistema más extendido en los centros educativos españoles, de modo que despacharlo en una línea sería ignorar el escenario más probable. Conviene separar dos papeles que se confunden con

facilidad.

Como cliente no hay nada que hacer. Los equipos del alumnado, y cualquier pantalla del aula que solo muestre subtítulos, funcionan con Windows exactamente igual que con cualquier otro sistema: abren una dirección en el navegador. Este es el caso más frecuente en la práctica, y conviene decirlo explícitamente porque la advertencia siguiente puede leerse mal.

Como servidor no se ha probado, pero la vía existe. El sistema no se ha desplegado sobre Windows en este trabajo, y por tanto no se afirma que funcione. Sí puede describirse con precisión qué haría falta, porque el obstáculo está localizado.

El camino sería el subsistema Linux de Windows, que hoy permite acceso a la tarjeta gráfica desde dentro. Tanto la plataforma de NVIDIA (Microsoft, 2024) como la de AMD (AMD, 2025) exponen la tarjeta a través de un dispositivo propio del subsistema, `/dev/dxg`, en lugar de los dispositivos que se usan en un Linux nativo. Y ahí está justamente el problema: **el perfil de AMD de este proyecto pasa al contenedor `/dev/kfd` y `/dev/dri`, que bajo el subsistema no existen.** Ese perfil requeriría por tanto modificarse. El de NVIDIA está mejor situado, porque no enumera dispositivos a mano sino que delega en el conjunto de herramientas del fabricante, que es precisamente el que se encarga de exponer el dispositivo correcto dentro del contenedor.

Hay un segundo obstáculo, menor pero real: los contenedores comparten la red del anfitrión, decisión tomada por un problema concreto de la distribución de desarrollo (sección C.2). Ese modo se comporta de forma distinta en el entorno de Docker para Windows, así que habría que volver a publicar puertos, que es lo que se hace normalmente.

Queda una tercera vía que conviene no perder de vista, porque puede ser la más simple: **ejecutar sin contenedores.** Los dos componentes del sistema son de por sí multiplataforma: la biblioteca de cálculo del servicio de reconocimiento tiene versión para Windows con tarjetas NVIDIA, y el entorno de ejecución de la aplicación es de Microsoft y nativo allí. Lo que no es multiplataforma es la automatización del proyecto, que asume un intérprete de órdenes de tipo Unix; habría que ejecutar los comandos a mano. Tampoco se ha probado.

En resumen, y para que un centro sepa a qué atenerse: sobre Windows con tarjeta NVIDIA el despliegue es plausible y probablemente cercano, sobre Windows con tarjeta AMD exige modificar el descriptor, y en ambos casos habría que verificarlo antes de comprometerse. Queda recogido como tarea pendiente en lugar de como afirmación.

El caso de Apple Silicon

Merece explicación aparte porque es contraintuitivo y porque un centro con equipamiento Apple lo encontrará de inmediato. Afecta a **toda la gama con chips de la serie M**, tanto portátiles como equipos de sobremesa, porque lo que lo provoca es la arquitectura del procesador y no el formato del equipo.

Conviene señalarlo porque los equipos de sobremesa de esa gama son, de hecho, candidatos razonables al papel de servidor de aula: son silenciosos, consumen poco y están pensados para permanecer encendidos, que es exactamente lo que este sistema necesita. Un centro que ya disponga de uno tiene una opción sensata, siempre que lo despliegue por la vía correcta.

Y la vía correcta no es la que parecería. La aplicación funciona sin cambios en esa arquitectura, dentro o fuera de contenedor; el problema está en el servicio de reconocimiento.

Ejecutado de forma nativa, el servicio detecta el acelerador integrado en el chip y lo emplea, con una particularidad que el código resuelve solo: fuerza precisión simple, porque Metal no implementa la precisión reducida para todas las operaciones del modelo y con ella la inferencia falla o devuelve silencio.

Ejecutado en contenedor, en cambio, **queda en procesador y no hay forma de evitarlo**. La causa no es del sistema desarrollado: en macOS los contenedores corren dentro de una máquina virtual Linux que no expone el acelerador de Apple. Y quedarse en procesador significa 1,6 veces la velocidad del audio (sección 5.6.5), insuficiente para directo. De ahí que en estos equipos la vía correcta sea la nativa, aunque sea la menos cómoda.

Queda una cosa por declarar, porque afecta a cuánta confianza merece esta recomendación: **el rendimiento de ese acelerador no se ha medido en este trabajo**. Se ha verificado que el servicio lo detecta y transcribe correctamente, pero el barrido de dispositivos comparó únicamente tarjeta gráfica dedicada y procesador (sección 5.6.5). Además, dentro de la propia gama el rendimiento varía mucho: no es comparable el chip de entrada de un portátil con las variantes de gama alta de un sobremesa, que multiplican el número de núcleos gráficos. Un centro que vaya a desplegar sobre este equipamiento debería medir el factor de tiempo real en su equipo concreto, con el mismo procedimiento del experimento, antes de comprometerse.

Estado de verificación de cada vía

Por el mismo motivo se declara aquí, y no en una nota al pie, qué está probado y qué no:

- **Vía nativa sobre Linux con tarjeta AMD:** es la empleada durante todo el desarrollo y la que produjo las mediciones de esta memoria. Probada.
- **Vía en contenedores sobre Linux con tarjeta AMD: construida y ejecutada.** Los dos contenedores arrancan, superan su comprobación de salud y se mantienen en servicio; las cifras de ocupación en disco de la tabla C.4 proceden de ese despliegue. Lo que no se ha hecho es una sesión de aula completa a través de ellos.
- **Vía en contenedores sobre Linux con tarjeta NVIDIA:** sin probar, por no disponer de una tarjeta de ese fabricante. El descriptor está escrito y difiere del anterior solo en la imagen base y en el mecanismo de acceso a la tarjeta.
- **Vía nativa sobre Apple Silicon:** el servicio detecta el acelerador y transcribe. Sin medición de rendimiento.
- **Windows, en cualquier variante:** sin probar como servidor. Como cliente funciona igual que cualquier otro sistema.

Este inventario es deliberado, y su nivel de detalle también. El capítulo 5 documenta varios casos en los que algo funcionaba en apariencia y producía resultados falsos; dar por verificada una vía de despliegue que nadie ha ejecutado sería exactamente el mismo error, y darla por no verificada cuando sí se ha ejecutado tampoco ayuda a quien tenga que desplegarla.

C.3. Puesta en marcha

Paso 1: obtener el proyecto

Basta con el repositorio; no hay que descargar el modelo a mano, porque el contenedor lo hace en el primer arranque y lo conserva.

Paso 2: configurar la clave del puesto docente

Se define en un fichero `.env` junto al descriptor de los contenedores:

`CLAVE_DOCENTE=la-clave-que-elija-el-centro`

Este paso no es opcional, y el descriptor de contenedores lo impone. Sin clave, la vista de emisión queda abierta y cualquier alumno conectado a la red podría tomarla y emitir por el aula. Durante un tiempo la variable tenía el valor vacío por defecto, de modo que seguir estos pasos sin definirla levantaba el sistema desprotegido sin ninguna advertencia; ahora el arranque se detiene y explica qué falta. La interfaz sigue avisando además cuando la aplicación corre sin clave, que es el modo de desarrollo.

En el mismo fichero conviene fijar la política de reinicio:

`REINICIO=unless-stopped`

Con ella, los contenedores vuelven a levantarse solos cuando arranca el equipo, que es lo que se quiere de un servidor de aula: tras un corte de luz o un reinicio nocturno, el sistema está disponible sin que nadie tenga que entrar a arrancarlo. El valor por defecto es el contrario, **no**, porque en un equipo de desarrollo unos contenedores que resucitan en cada encendido son una molestia, y el del reconocimiento además mantiene ocupado un núcleo y varios gigabytes de memoria mientras vive.

Se emplea `unless-stopped` y no `always` por una diferencia que importa en un centro: si alguien detiene el sistema a propósito, debe seguir detenido hasta que se decida arrancarlo, y no reaparecer en el siguiente encendido.

Paso 3: levantar el sistema

Un único comando, elegido según la tabla C.5:

<code>make docker-amd</code>	(Linux, tarjetas AMD, plataforma ROCm)
<code>make docker-nvidia</code>	(Linux, tarjetas NVIDIA, plataforma CUDA)
<code>make demo</code>	(macOS con Apple Silicon, ejecución nativa)

Entre las dos primeras la diferencia es únicamente la imagen base del servicio de reconocimiento; el código es idéntico, porque la biblioteca de cálculo expone la misma interfaz sobre las dos plataformas. La tercera no usa contenedores por el motivo explicado en la sección C.2, y requiere tener preparado el entorno de Python y el de la aplicación en el propio equipo.

El primer arranque es lento y eso es normal. Hay que descargar la imagen base, que en el caso de AMD son más de ochenta gigabytes (tabla C.4), descargar el modelo

y compilar los núcleos de cálculo de la tarjeta. Todo ello queda guardado en volúmenes, de modo que los arranques siguientes son inmediatos. Conviene por tanto hacer el primer arranque con un día de margen sobre la primera clase, y con una conexión que aguante esa descarga.

Paso 4: comprobar que funciona

El comando anterior imprime al arrancar las dos direcciones del sistema, recogidas en la tabla C.7. Son distintas y no son intercambiables:

Quién	Dirección	Observación
Docente	<code>http://localhost:5203/broadcast</code>	Debe abrirse en el propio equipo servidor .
Alumnado	<code>http://<ip-del-equipo>:5203/view</code>	Desde cualquier dispositivo de la misma red.

Tabla C.7: Direcciones de acceso al sistema.

La restricción de la primera fila tiene una causa concreta y conviene entenderla antes de intentar sortearla: los navegadores solo conceden acceso al micrófono en conexiones seguras o en la máquina local. Como el sistema opera dentro de la red del centro y sin certificado, **la emisión tiene que hacerse desde el equipo servidor**. Si se quisiera emitir desde otro equipo del aula habría que servir la aplicación por HTTPS con un certificado reconocido por los dispositivos del centro, lo que es perfectamente posible pero excede lo que este trabajo despliega.

Paso 5: dar clase

En la vista del docente: introducir la clave, pegar en el recuadro del glosario los términos de la sesión, uno por línea, y pulsar el botón de comenzar. Los subtítulos aparecen a la vez en la pantalla del docente y en la de todo el alumnado conectado.

El glosario conviene tomarlo del temario o de las transparencias de la sesión. No hace falta que sea exhaustivo: sirve para que los términos propios de la asignatura se resalten visualmente en el subtítulo.

Paso 6: detener el sistema

```
make docker-parar
```

Detiene los contenedores sin borrar el modelo descargado ni las cachés, de modo que el arranque siguiente vuelva a ser inmediato.

C.4. Incorporar el modelo adaptado

El resultado del ajuste fino se activa añadiendo una línea al mismo fichero `.env`:

```
ADAPTADOR=/adaptador
```

No hay que reconstruir nada ni modificar la aplicación: el servicio de reconocimiento aplica el adaptador sobre el modelo base al cargarlo. Dejar la variable vacía sirve el modelo sin adaptar. Es la comprobación práctica de la frontera descrita en la sección 5.6.12.

C.5. Qué hacer si algo no funciona

Los tres fallos que se han observado en la práctica tienen la misma característica que los incidentes del capítulo 5: el sistema no se cae, simplemente funciona peor sin decirlo.

Los subtítulos tardan mucho y llegan en bloques largos. Mirar el contador de la vista del docente que separa los cortes cerrados *por pausa* de los cerrados *por agotar el tope*. Si dominan los segundos, el detector de silencios no está encontrando las pausas. La causa habitual no está en el sistema sino en el navegador: el control automático de ganancia amplifica el ruido de fondo cuando el hablante calla, y la energía nunca baja lo suficiente. La aplicación lo desactiva al capturar; si aun así ocurre, revisar que no haya un procesado de audio activo en el sistema operativo o en el propio micrófono.

Aparecen frases que nadie ha dicho. El sistema filtra los dos patrones reconocibles, las muletillas de subtítulos de vídeo y los bucles de repetición, pero no puede filtrar una alucinación verosímil (sección 5.6.7). Si son frecuentes, casi siempre indica que al micrófono le llega poca voz y mucho fondo: acercarlo al docente antes que tocar ninguna configuración.

La calidad cae de golpe a mitad de clase. La degradación frente al ruido no es gradual: el sistema pasa de funcionar bien a fallar en un margen de cinco decibelios (sección 5.6.6). Si el aula se ha animado, el remedio es el micrófono de proximidad, no un ajuste del software.

El contenedor de reconocimiento no arranca con tarjeta AMD. Casi siempre son los identificadores de los grupos `video` y `render`, que varían entre equipos y están

fijados numéricamente en el descriptor. Se consultan con `getent group video render` y se ajustan.

C.6. Arranque sin contenedores

Los dos atajos que se emplearon durante el desarrollo sirven también para desplegar, y son la vía obligada en macOS:

```
make app      arranca solo la aplicación, con un motor de reconocimiento si-
mulado
make demo     arranca el servicio de reconocimiento real y la aplicación, y
limpia al salir
make demo-lora igual, pero sirviendo el modelo adaptado
```

El primero merece atención porque resuelve un problema práctico de despliegue: permite comprobar que la red del aula, los dispositivos del alumnado y las direcciones de acceso funcionan *antes* de que entre en juego el reconocimiento. Si los subtítulos simulados llegan a los móviles de la clase y los reales no, el problema está en el servicio de reconocimiento; si no llegan ni los simulados, está en la red. Separar esas dos causas ahorra la mayor parte del tiempo de diagnóstico.